

Recursion

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Recursion	1
1.1	The One Question Every Recursive Function Must Answer	1
1.2	Count Good Numbers	3
1.3	Implement Pow(x, n)	5
1.4	Recursive Implementation of atoi	7
1.5	Reverse a Stack using Recursion	9
1.6	Sort a Stack using Recursion	12
1.7	Power Set: Print All Subsequences	14
1.8	Check if There Exists a Subsequence with Sum K	16
1.9	Count Subsequences with Sum K	18
1.10	Subset Sums I (Print All Subset Sums)	20
1.11	Subset Sums II (Unique Subsets with Duplicates Present)	22
1.12	Combination Sum	25
1.13	Combination Sum II	27
1.14	Combination Sum III	30
1.15	Generate Parentheses	32
1.16	Generate All Binary Strings with No Two Consecutive 1s	34
1.17	Letter Combinations of a Phone Number	36
1.18	Rat in a Maze	39
1.19	N-Queens	41
1.20	Sudoku Solver	44
1.21	Word Search	47
1.22	Palindrome Partitioning	49
1.23	Word Break	52

Chapter 1

Recursion

To understand recursion, trust the recursion. Assume your function already correctly solves every strictly smaller version of the problem, and ask only one question: given those correct smaller answers, how do I build the answer to this input? If you can answer that question once, you have solved every input, by induction.

1.1 The One Question Every Recursive Function Must Answer

Every recursive function is built from exactly three ingredients: a **base case** (the smallest input(s) the function can answer directly, without recursing), a **recursive case** (how to build this input's answer from one or more strictly smaller sub-calls, trusted to already be correct), and a guarantee that the sub-calls' inputs actually *shrink* toward the base case (otherwise the recursion never terminates).

The Recursive Leap of Faith

When designing a recursive function, do not try to mentally trace through every recursive call – this is the single most common source of confusion and wasted time. Instead, write down what the function is *supposed* to do in words, assume every call with a strictly smaller input already does exactly that correctly, and then write only the logic needed to combine those trusted smaller answers into this input's answer. Verifying the base case and the combination step is enough to prove the whole function correct, by mathematical induction on the input's size – there is no need to simulate the call stack by hand except when actively debugging.

Two Families of Recursion in This Chapter

How to Recognise Which Family You Are In

- “**Compute a single value/result**” (a power, a digit count, a stack reversed or sorted) – **straight-line recursion**: each call makes exactly one (or a small fixed number of) recursive call(s) and combines the result(s) in a fixed way. The skill here is finding the right way to shrink the input and the right combination step, often exploiting a mathematical identity (as in fast exponentiation) to shrink faster than one unit at a time.
- “**Generate/find all ...**” (subsequences, subsets, combinations, permutations, paths, valid placements) – **exploration recursion**, commonly called **backtracking**: each call branches into *multiple* recursive calls, one per choice available at this step, and an explicit *undo* step restores shared state after each branch returns, so that the next branch starts from a clean slate. The skill here is identifying, at each step, the full set of choices available, and recognizing when a partial choice can be abandoned early (pruned) because it can no longer lead to a valid complete answer.

This chapter moves from the first family (Section 1.2 through Section 1.6) into the second

(Section 1.7 onward), building backtracking’s “choose, explore, un-choose” template up gradually through string and subset generation before tackling full constraint-satisfaction problems like N-Queens and Sudoku.

Why Recursion Sets Up Everything in the Dynamic Programming Chapters

Every dynamic programming problem in the chapters that follow *begins* as a recursive solution: identify the choices available at each step, write the recursive recurrence that combines smaller sub-answers into this one, and verify it against small examples by hand – exactly the discipline built in this chapter. Memoization and tabulation, covered there, are mechanical transformations applied *after* a correct recursive solution already exists; they do not replace the need to find that recursion first. Mastering straight-line and exploration recursion here is therefore not a detour before dynamic programming – it is dynamic programming’s first and most important step, practiced in isolation.

Each section in this chapter follows the established structure: problem statement and keywords; the reasoning that identifies the base case, the recursive case, and (for backtracking problems) the choice set and any pruning opportunity; the algorithm in words and pseudocode; a hand-traced dry run (often shown as a small recursion tree); complete compilable C++ code; and a time/space complexity analysis, including the recursion’s call-stack depth.

1.2 Count Good Numbers

Problem Statement. A digit string of length n is called **good** if every digit at an **even** index (0-indexed) is one of the even digits $\{0, 2, 4, 6, 8\}$, and every digit at an **odd** index is one of the prime digits $\{2, 3, 5, 7\}$. Count how many good digit strings of length n exist, modulo $10^9 + 7$.

Pattern Recognition

Despite involving “counting all possibilities,” this is *not* a backtracking problem – there is no need to actually generate any digit string, because each position’s choice is completely **independent** of every other position’s choice. Whenever every position in a sequence can be filled independently from a fixed-size choice set, the total count is simply the **product** of each position’s choice count, and that product is best computed via **fast exponentiation**, a straight-line recursion that shrinks an exponent by half at each step.

Understanding the problem carefully

Among the n positions, $\lceil n/2 \rceil$ are at even indices $(0, 2, 4, \dots)$ and $\lfloor n/2 \rfloor$ are at odd indices. Each even-index position independently has 5 valid choices $(0, 2, 4, 6, 8)$, and each odd-index position independently has 4 valid choices $(2, 3, 5, 7)$. By the multiplication principle (every combination of independent choices is a distinct valid string), the total count is

$$5^{\lceil n/2 \rceil} \times 4^{\lfloor n/2 \rfloor} \pmod{10^9 + 7}.$$

The only remaining challenge is computing a large power modulo $10^9 + 7$ efficiently, since n can be large enough that computing $5^{n/2}$ by repeated multiplication, one factor at a time, would be too slow.

Fast Exponentiation: Halve the Exponent, Not Decrement It

To compute $base^{exp}$: if $exp = 0$, the answer is 1 (base case). If exp is even, recursively compute $half = base^{exp/2}$, and the answer is $half \times half$. If exp is odd, compute $half = base^{(exp-1)/2}$, and the answer is $half \times half \times base$.

Why halving the exponent gives $O(\log n)$ instead of $O(n)$

A naive recursive power function that decrements the exponent by 1 at each call would need $O(n)$ recursive calls, each doing one multiplication. Halving the exponent instead means the recursion depth is only $O(\log n)$ – each level squares an already-computed result rather than multiplying by the base value just once, exploiting the identity $base^{2k} = (base^k)^2$ to skip directly from computing one power to a power roughly twice as large, in a single multiplication. This is the same “shrink faster than one unit at a time” idea elaborated fully in Section 1.3.

Algorithm

1. $evenCount \leftarrow \lceil n/2 \rceil$, $oddCount \leftarrow \lfloor n/2 \rfloor$.
2. Compute $a \leftarrow 5^{evenCount} \pmod{10^9 + 7}$ via fast exponentiation.
3. Compute $b \leftarrow 4^{oddCount} \pmod{10^9 + 7}$ via fast exponentiation.
4. Return $(a \times b) \pmod{10^9 + 7}$.

Dry run

Dry Run. Take $n = 4$.

$evenCount = 2$ (indices 0,2), $oddCount = 2$ (indices 1,3). $5^2 = 25$, $4^2 = 16$. Answer $= 25 \times 16 = 400$.

Checking by hand for a tiny case, $n = 1$: $evenCount = 1$, $oddCount = 0$. $5^1 = 5$, $4^0 = 1$. Answer $= 5$ – correctly, the 5 single-digit good numbers are 0, 2, 4, 6, 8 (index 0 is even, so any even digit qualifies, and there is no odd index to satisfy).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 long long power(long long base, long long exp) {
6     if (exp == 0) return 1;
7     long long half = power(base, exp / 2) % MOD;
8     long long result = (half * half) % MOD;
9     if (exp % 2 == 1) result = (result * base) % MOD;
10    return result;
11 }
12
13 int countGoodNumbers(long long n) {
14     long long evenCount = (n + 1) / 2;
15     long long oddCount = n / 2;
16
17     long long a = power(5, evenCount);
18     long long b = power(4, oddCount);
19     return (a * b) % MOD;
20 }
21
22 int main() {
23     cout << "Good numbers of length 4: " << countGoodNumbers(4) << endl
24         ;
25     return 0;
26 }
```

Complexity Analysis

Time Complexity. $O(\log n)$ for each fast-exponentiation call, giving an overall time complexity of

$$O(\log n).$$

Space Complexity. $O(\log n)$ for the recursion stack of fast exponentiation.

Key Takeaway

Whenever a counting problem's choices at every position are independent of each other, resist the urge to enumerate – multiply the per-position choice counts directly, and reach for fast exponentiation the moment that product involves a single base raised to a large power. This pattern of recognizing independence to avoid generation entirely recurs throughout combinatorial counting problems.

1.3 Implement Pow(x, n)

Problem Statement. Implement a function that computes x^n for a real number x and an integer n , which may be negative, in better than $O(|n|)$ time.

Pattern Recognition

“Compute x^n efficiently” is the canonical introduction to **fast exponentiation** (also called binary exponentiation), already used as a building block in Section 1.2. The keyword to internalize generally: whenever a recursive shrinking step can **halve** the input rather than merely decrement it by one, the recursion depth drops from $O(n)$ to $O(\log n)$ – a pattern that reappears constantly, not just for exponentiation.

Understanding the problem carefully

The naive recursive definition $x^n = x \times x^{n-1}$ (with base case $x^0 = 1$) is correct but requires n recursive calls. The key algebraic identity that lets us do better is

$$x^n = \left(x^{n/2}\right)^2 \quad \text{when } n \text{ is even,} \quad x^n = x \times \left(x^{(n-1)/2}\right)^2 \quad \text{when } n \text{ is odd.}$$

Either way, computing x^n only requires computing *one* smaller power, $x^{\lfloor n/2 \rfloor}$, and then a constant amount of extra multiplication – the exponent is halved at every recursive step instead of merely decremented.

Halve the Exponent, Handle Negative Exponents Separately

If $n = 0$: return 1 (base case). If $n < 0$: compute x^{-n} using the same recursion, and return its reciprocal $1/x^{-n}$ at the very end (do not try to handle negative exponents inside the recursive halving step itself – convert once, up front, and recurse only on non-negative exponents). For $n > 0$: recursively compute $half = x^{\lfloor n/2 \rfloor}$; if n is even, return $half \times half$; if n is odd, return $half \times half \times x$.

Why halving gives logarithmic depth

Each recursive call’s exponent is at most half of its caller’s exponent, so after k calls the exponent has shrunk by a factor of 2^k – it reaches the base case 0 once $2^k \geq n$, i.e. after only $O(\log n)$ calls, regardless of how large n is. This is in sharp contrast to the naive decrement-by-one recursion, whose depth is exactly n . The trade-off is that each call here does a small constant amount of extra multiplication work (squaring the smaller result, and possibly one more multiplication by x for odd exponents), but this constant overhead is negligible compared to the exponential reduction in the number of calls needed.

Algorithm

1. If $n < 0$: return $1/\text{Pow}(x, -n)$.
2. $\text{Pow}(x, n)$:
 - (a) If $n = 0$: return 1.
 - (b) $half \leftarrow \text{Pow}(x, n/2)$ (integer division).
 - (c) If n is even: return $half \times half$.
 - (d) Else: return $half \times half \times x$.

Dry run

Dry Run. Compute 2^{10} .

- Pow(2, 10): 10 even. $half = \text{Pow}(2, 5)$.
- Pow(2, 5): 5 odd. $half = \text{Pow}(2, 2)$.
- Pow(2, 2): 2 even. $half = \text{Pow}(2, 1)$.
- Pow(2, 1): 1 odd. $half = \text{Pow}(2, 0) = 1$. Return $1 \times 1 \times 2 = 2$.
- Back at Pow(2, 2): $half = 2$. Return $2 \times 2 = 4$.
- Back at Pow(2, 5): $half = 4$. Return $4 \times 4 \times 2 = 32$.
- Back at Pow(2, 10): $half = 32$. Return $32 \times 32 = 1024$.

$2^{10} = 1024$, confirmed. Only 4 recursive calls were needed for an exponent of 10, instead of 10 calls a naive decrement-by-one recursion would have required.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  double fastPow(double x, long long n) {
5      if (n == 0) return 1.0;
6      double half = fastPow(x, n / 2);
7      double result = half * half;
8      if (n % 2 == 1) result *= x;
9      return result;
10 }
11
12 double myPow(double x, int n) {
13     long long N = n;
14     if (N < 0) return 1.0 / fastPow(x, -N);
15     return fastPow(x, N);
16 }
17
18 int main() {
19     cout << myPow(2.0, 10) << endl;    // 1024
20     cout << myPow(2.0, -2) << endl;    // 0.25
21     return 0;
22 }
```

Complexity Analysis

Time Complexity.

$O(\log n)$.

Space Complexity. $O(\log n)$ for the recursion stack (or $O(1)$ if rewritten as an iterative loop that repeatedly squares the base and halves the exponent, accumulating a result whenever the current bit of the exponent is set).

Key Takeaway

Fast exponentiation is the prototype for an entire family of techniques: whenever an operation is associative (like multiplication) and you need to apply it n times, check whether the operation can be “doubled up” (squared, in this case) to cut the required number of steps from $O(n)$ to $O(\log n)$. The same halving idea underlies fast matrix exponentiation for linear recurrences and binary lifting for tree ancestor queries, both common extensions of this exact pattern.

1.4 Recursive Implementation of atoi

Problem Statement. Implement (recursively) the C-style *atoi* function: given a string, skip any leading whitespace, then an optional + or - sign, then consume as many leading digits as possible, and return the resulting integer – clamped to the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$ if it would otherwise overflow. If no valid number can be parsed at all, return 0.

Pattern Recognition

This problem has no clever algorithmic trick – the difficulty is purely in correctly handling several small edge cases (whitespace, sign, leading zeros, early non-digit characters, overflow) using recursion instead of an iterative loop. The keyword to notice: “*recursive implementation of*” an otherwise simple iterative task is an exercise in expressing a left-to-right scan as a chain of recursive calls, each processing one character and delegating the rest to a sub-call.

Understanding the problem carefully

The parsing naturally splits into three independent phases, each best handled by its own small recursive (or simple iterative) helper: skipping whitespace, consuming an optional sign, and then accumulating digits one at a time. The digit-accumulation phase is the one most naturally expressed recursively: at each step, if the current character is a digit, fold it into a running accumulated value and recurse on the rest of the string; the moment a non-digit (or the string’s end) is reached, stop and return the accumulated value so far.

An Accumulator Passed Down Through the Recursion

`PARSEDIGITS( $s, i, acc$ )`: if i is out of bounds or $s[i]$ is not a digit, return acc (base case – nothing more to consume). Otherwise, compute $newAcc = acc \times 10 + (s[i] - '0')$, clamp $newAcc$ immediately if it has already exceeded the representable range (to avoid undefined-behavior overflow in the accumulator itself), and recurse: `PARSEDIGITS( $s, i + 1, newAcc$ )`.

Why clamping during accumulation, not just at the end, matters

If the accumulator is allowed to grow without bound while scanning a string of many digits (e.g. a string of 50 digits), it can overflow even a 64-bit accumulator long before the recursion finishes, leading to incorrect (wrapped-around) results rather than a clean clamp. Checking and clamping the accumulator the moment it *exceeds* the valid 32-bit range – and, once clamped, simply continuing to consume (and ignore) any remaining digits so the function still terminates at the string’s actual end of digits – avoids this, ensuring the final returned value is always either the true parsed integer or correctly clamped to $\pm 2^{31} - 1$.

Algorithm

1. Skip leading whitespace characters (advance an index i while $s[i]$ is a space).
2. If $s[i]$ is $+$ or $-$: record the sign, advance i .
3. Call PARSEDIGITS($s, i, 0$) to accumulate the digit value, clamping along the way as described above.
4. Apply the recorded sign to the result, then clamp the signed result to $[-2^{31}, 2^{31} - 1]$ one final time.

Dry run

Dry Run. Take $s = " -042abc"$.

- Skip whitespace: i advances past the two leading spaces.
- Sign: $s[i] = '-'$, record negative, advance i .
- PARSEDIGITS: consume 0 ($acc = 0$), 4 ($acc = 4$), 2 ($acc = 42$); next character is **a**, not a digit – stop, return 42.
- Apply sign: -42 . Well within range, no clamping needed.

Result: -42 .

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long parseDigits(const string& s, int i, long long acc) {
5      if (i >= (int)s.size() || !isdigit(s[i])) return acc;
6
7      long long newAcc = acc * 10 + (s[i] - '0');
8      if (newAcc > INT_MAX) newAcc = (long long)INT_MAX + 1; // clamp
9      return parseDigits(s, i + 1, newAcc);
10 }
11
12 int myAtoi(string s) {
13     int i = 0, n = s.size();
14     while (i < n && s[i] == ' ') i++;
15
16     int sign = 1;
17     if (i < n && (s[i] == '+' || s[i] == '-')) {
18         if (s[i] == '-') sign = -1;
19         i++;
20     }
21
22     long long result = parseDigits(s, i, 0) * sign;
23
24     if (result > INT_MAX) return INT_MAX;
25     if (result < INT_MIN) return INT_MIN;
26     return (int)result;
27 }
28

```

```

29 int main() {
30     cout << myAtoi("  -042abc") << endl;           // -42
31     cout << myAtoi("91283472332") << endl;         // 2147483647 (
        clamped)
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. $O(L)$, where L is the length of the input string – each character is examined at most once across the whitespace-skip, sign-check, and digit-accumulation phases. **Space Complexity.** $O(L)$ for the digit-accumulation recursion stack in the worst case (a string of all digits); this could be reduced to $O(1)$ by rewriting the accumulation phase as an iterative loop, which is how production implementations typically do it.

Key Takeaway

Not every recursive exercise hides a deep algorithmic insight – some, like this one, exist purely to practice expressing a sequential scan (skip, check, accumulate, stop) as a chain of base-case-and-recurse calls, with an accumulator threaded through as an explicit parameter rather than a mutable loop variable. This accumulator-passing style is the same technique used for tail-recursive functions in functional programming.

1.5 Reverse a Stack using Recursion

Problem Statement. Given a stack, reverse its order using **only** the stack’s own *push* and *pop* operations and **recursion** – no second stack, array, or queue is allowed as auxiliary storage.

Pattern Recognition

“Solve using only recursion, no auxiliary data structure” is a direct instruction to use the **call stack itself** as the only permitted extra storage: every value temporarily removed from the stack must be held as a local variable on some recursive call’s stack frame, not in any explicit container. This requires a second, helper recursive function – inserting a value back at the *bottom* of a stack is itself a small recursive sub-problem, since a stack has no native “insert at the bottom” operation.

Understanding the problem carefully

The overall idea: pop the top element off, recursively reverse *everything else* below it, and then place the popped element back – but at the *bottom* of the now-reversed remainder, since reversal means the original top must end up at the very bottom of the final stack. Placing a value at the bottom of a stack, using only *push/pop*, is itself solved recursively: pop everything off one at a time, push the target value once the stack is empty, then push everything else back on top, in the reverse of the order it was removed (which recursion handles automatically via the call stack’s own LIFO unwinding).

Two Cooperating Recursive Functions

INSERTATBOTTOM(*stack*, *x*): if *stack* is empty, push *x* and return (base case). Otherwise, pop the top into a local variable *temp*, recursively call INSERTATBOTTOM(*stack*, *x*) (placing *x* at the bottom of what remains), then push *temp* back on top. REVERSESTACK(*stack*): if *stack* is empty (or has one element), return (base case). Otherwise, pop the top into

temp, recursively call REVERSESTACK(*stack*) (reversing everything below *temp*), then call INSERTATBOTTOM(*stack*, *temp*).

Why local variables on the call stack correctly substitute for an explicit auxiliary structure

Each recursive call to REVERSESTACK holds exactly one popped element in its own local variable *temp*, alive for the duration of that call (including everything that happens inside its recursive sub-call). Because the call stack itself maintains these local variables in a strict LIFO order matching the order elements were popped, and because each element is pushed back (via INSERTATBOTTOM) exactly once as its corresponding call returns, no information is lost and no second container is ever needed – the recursion’s own bookkeeping does the job an explicit auxiliary stack would otherwise have done.

Algorithm

1. INSERTATBOTTOM(*stack*, *x*):
 - (a) If *stack* is empty: push *x*; return.
 - (b) Pop top into *temp*.
 - (c) INSERTATBOTTOM(*stack*, *x*).
 - (d) Push *temp*.
2. REVERSESTACK(*stack*):
 - (a) If *stack* is empty: return.
 - (b) Pop top into *temp*.
 - (c) REVERSESTACK(*stack*).
 - (d) INSERTATBOTTOM(*stack*, *temp*).

Dry run

Dry Run. Reverse the stack [1, 2, 3] (top to bottom: 3, 2, 1).

- REVERSESTACK call 1: pop 3. Recurse on [1, 2].
- Call 2: pop 2. Recurse on [1].
- Call 3: pop 1. Recurse on [] (empty, base case, returns immediately).
- Back in call 3: INSERTATBOTTOM([], 1): stack is empty, push 1. Stack = [1].
- Back in call 2: INSERTATBOTTOM([1], 2): pop 1 into *temp*; recurse INSERTATBOTTOM([], 2): empty, push 2, stack= [2]; back in outer call, push *temp* (1) back: stack= [2, 1] (top to bottom: 1, 2).
- Back in call 1: INSERTATBOTTOM([2, 1], 3): pop 1 into *temp*; recurse on [2]: pop 2 into another *temp*; recurse on []: push 3, stack= [3]; push back the inner *temp* (2): stack= [3, 2]; push back the outer *temp* (1): stack= [3, 2, 1].

Final stack (top to bottom): 1, 2, 3 – the reverse of the original 3, 2, 1.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void insertAtBottom(stack<int>& st, int x) {
5      if (st.empty()) {
6          st.push(x);
7          return;
8      }
9      int temp = st.top();
10     st.pop();
11     insertAtBottom(st, x);
12     st.push(temp);
13 }
14
15 void reverseStack(stack<int>& st) {
16     if (st.empty()) return;
17
18     int temp = st.top();
19     st.pop();
20     reverseStack(st);
21     insertAtBottom(st, temp);
22 }
23
24 int main() {
25     stack<int> st;
26     st.push(1); st.push(2); st.push(3);
27
28     reverseStack(st);
29
30     while (!st.empty()) {
31         cout << st.top() << " ";
32         st.pop();
33     }
34     cout << endl;
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. INSERTATBOTTOM costs $O(k)$ for a stack of size k (it must pop and re-push every element to reach the bottom). REVERSESTACK calls it once for each of the n elements, with the stack shrinking by one each time, giving

$$O(1 + 2 + \dots + n) = O(n^2).$$

Space Complexity. $O(n)$ for the recursion stack at the deepest point (both functions' call depths are bounded by n).

Key Takeaway

“Using only recursion, no auxiliary data structure” is a direct instruction to use *local variables on the call stack* as your only permitted temporary storage. Many such “stack-only” problems (reversing, sorting, this section and the next) need exactly this kind of helper function that performs an operation a real stack does not natively support (here, inserting

at the bottom), built entirely out of recursive pop/push/recurse/push-back steps.

1.6 Sort a Stack using Recursion

Problem Statement. Given a stack, sort it in increasing order (so the smallest element ends up on top) using **only** *push* and *pop*, and recursion – no second stack, array, or queue allowed.

Pattern Recognition

This is Section 1.5’s “insert at the bottom” helper, generalized to “insert at the *correct sorted position*.” The overall structure – pop the top, recursively solve the rest, then reinsert the popped value using a specialized helper – is identical; only the helper’s insertion rule changes from “always go to the bottom” to “go just above everything smaller, and below everything larger.”

Understanding the problem carefully

If the remainder of the stack (everything below the current top) is already correctly sorted, inserting the popped-off top element into its correct position relative to that sorted remainder finishes the job – this is exactly insertion sort’s core idea, expressed with a stack playing the role that an array’s shifting-elements-right step normally plays.

Insert into a Sorted Stack at the Correct Position

INSERTSORTED(*stack*, *x*): if *stack* is empty *or* its top is $\leq x$, push *x* and return (its correct position has been found: everything remaining below is $\leq x$, satisfying sorted order with the top of the stack as the largest visible value at this point... more precisely, the stack is kept increasing from top to bottom, so *x* belongs here once it is at least as large as what is currently on top). Otherwise, pop the top into *temp*, recursively call INSERTSORTED(*stack*, *x*) (continue searching deeper for *x*’s correct position), then push *temp* back on top.

Why this insertion rule keeps the stack sorted at every step

Assume, inductively, that before inserting *x*, the stack is already sorted (say, increasing from top to bottom: the smallest element on top). The helper pops elements off only while they are *larger* than *x* – precisely the elements that must end up *above* *x* once *x* is correctly placed. The moment it finds an element $\leq x$ (or the stack empties), that is exactly the right depth to insert *x*, and pushing the previously-popped larger elements back afterward restores them directly on top of *x*, in their original relative order (preserved by the recursive call stack’s own LIFO unwinding) – so the stack remains sorted after the insertion completes.

Algorithm

1. INSERTSORTED(*stack*, *x*):
 - (a) If *stack* is empty or top $\leq x$: push *x*; return.
 - (b) Pop top into *temp*.
 - (c) INSERTSORTED(*stack*, *x*).
 - (d) Push *temp*.
2. SORTSTACK(*stack*):

- (a) If *stack* is empty: return.
- (b) Pop top into *temp*.
- (c) SORTSTACK(*stack*).
- (d) INSERTSORTED(*stack*, *temp*).

Dry run

Dry Run. Sort the stack [3, 1, 2] (top to bottom: 2, 1, 3).

- Pop 2. Recurse on [3, 1] (top to bottom: 1, 3).
- Pop 1. Recurse on [3].
- Pop 3. Recurse on [] (base case).
- INSERTSORTED([], 3): empty, push. Stack= [3].
- Back in the “pop 1” frame: INSERTSORTED([3], 1): top is $3 > 1$, pop 3 into *temp*; recurse INSERTSORTED([], 1): empty, push 1, stack= [1]; push back *temp* (3): stack= [1, 3] (top to bottom: 1, 3 – sorted).
- Back in the “pop 2” frame: INSERTSORTED([1, 3], 2): top is $1 \leq 2$, push 2 directly. Stack= [2, 1, 3] (top to bottom: 2, 1, 3).

Final stack, top to bottom: 1, 2, 3 – correctly sorted, smallest on top.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void insertSorted(stack<int>& st, int x) {
5      if (st.empty() || st.top() <= x) {
6          st.push(x);
7          return;
8      }
9      int temp = st.top();
10     st.pop();
11     insertSorted(st, x);
12     st.push(temp);
13 }
14
15 void sortStack(stack<int>& st) {
16     if (st.empty()) return;
17
18     int temp = st.top();
19     st.pop();
20     sortStack(st);
21     insertSorted(st, temp);
22 }
23
24 int main() {
25     stack<int> st;
26     st.push(3); st.push(1); st.push(2);
27
28     sortStack(st);

```

```

29
30     while (!st.empty()) {
31         cout << st.top() << " ";
32         st.pop();
33     }
34     cout << endl;
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. $O(n^2)$ in the worst case – the same insertion-sort-style bound as Section 1.5, since INSERTSORTED can pop and re-push up to $O(k)$ elements for a stack of size k , summed over n insertions. **Space Complexity.** $O(n)$ for the recursion stack at the deepest point.

Key Takeaway

Once you have solved one “insert at a specific position using only recursion” problem (insert at the bottom, in Section 1.5), solving a closely related one is just a matter of changing the stopping condition inside the same recursive shape: stop unconditionally at the bottom for reversal, stop as soon as a smaller-or-equal element is found for sorting.

1.7 Power Set: Print All Subsequences

We now begin the chapter’s second family: **exploration recursion**, or backtracking. This first problem introduces the single template – the **pick/not-pick** decision tree – that every other problem in this group (Sections 1.7–1.17) specializes or extends.

Problem Statement. Given an array (or string) of n distinct elements, generate **every** subsequence (equivalently, every subset, since order within a subsequence here does not matter for counting purposes) – all 2^n of them, including the empty subsequence.

Pattern Recognition

“Generate all subsequences/subsets” is the foundational backtracking pattern: at every element of the input, there are exactly **two** choices – include it in the current subsequence, or don’t – and these two choices are completely independent across different elements. A recursion that branches into exactly these two calls at every position, one per element, visits every one of the 2^n combinations exactly once.

Understanding the problem carefully

Think of building a subsequence by walking through the input array left to right, making one binary decision per element: “is this element in my subsequence, or not?” After making this decision for every one of the n elements, exactly one specific subsequence has been fully determined. Since there are 2 choices at each of n independent positions, there are exactly 2^n distinct sequences of choices – matching exactly the 2^n subsequences a set of n elements has.

The Pick/Not-Pick Template

SOLVE($index, current$): if $index = n$ (every element has been decided): record $current$ as one complete subsequence; return (base case). Otherwise: **pick** – append $arr[index]$ to $current$, recurse with SOLVE($index + 1, current$), then **undo** the append (remove the last

element, restoring *current* to how it was before this branch). **Not-pick** – recurse with `SOLVE(index + 1, current)` unchanged (the element is simply skipped).

Why the explicit “undo” step after the pick branch is essential

If *current* is a single shared mutable list passed by reference into both branches (rather than a fresh copy made for each), then after the pick branch’s entire recursive exploration finishes and control returns to this level, *current* must be restored to exactly the state it had *before* the pick was made – otherwise the upcoming not-pick branch would incorrectly start from a list that still contains the picked element. This pattern – **choose**, recurse, **un-choose** – is the defining rhythm of every backtracking algorithm in this chapter; forgetting the un-choose step is the single most common bug when implementing backtracking with a shared, mutated data structure instead of fresh copies.

Algorithm

1. `SOLVE(index, current)`:
 - (a) If $\text{index} = n$: add a copy of *current* to the result list; return.
 - (b) Append `arr[index]` to *current*.
 - (c) `SOLVE(index + 1, current)`.
 - (d) Remove the last element of *current* (undo).
 - (e) `SOLVE(index + 1, current)` (not-pick branch, *current* unchanged from before step 2).
2. Call `SOLVE(0, [])` to start.

Dry run

Dry Run. Take $\text{arr} = [1, 2]$ ($n = 2$).

The recursion tree:

- `SOLVE(0, [])`:
 - Pick 1: `SOLVE(1, [1])`:
 - * Pick 2: `SOLVE(2, [1, 2])`: base case, record `[1, 2]`.
 - * Undo; Not-pick 2: `SOLVE(2, [1])`: base case, record `[1]`.
 - Undo; Not-pick 1: `SOLVE(1, [])`:
 - * Pick 2: `SOLVE(2, [2])`: base case, record `[2]`.
 - * Undo; Not-pick 2: `SOLVE(2, [])`: base case, record `[]`.

Recorded subsequences, in the order generated: `[1, 2]`, `[1]`, `[2]`, `[]` – all $2^2 = 4$ subsequences of `{1, 2}`, each exactly once.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(int index, vector<int>& arr, vector<int>& current,
5           vector<vector<int>>& result) {
6     if (index == (int)arr.size()) {
```

```

7         result.push_back(current);
8         return;
9     }
10
11     current.push_back(arr[index]);          // pick
12     solve(index + 1, arr, current, result);
13     current.pop_back();                    // undo
14
15     solve(index + 1, arr, current, result); // not-pick
16 }
17
18 vector<vector<int>> powerSet(vector<int>& arr) {
19     vector<vector<int>> result;
20     vector<int> current;
21     solve(0, arr, current, result);
22     return result;
23 }
24
25 int main() {
26     vector<int> arr = {1, 2};
27     for (auto& subset : powerSet(arr)) {
28         cout << "[ ";
29         for (int x : subset) cout << x << " ";
30         cout << "] ";
31     }
32     cout << endl;
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. There are 2^n leaves in the recursion tree (one per subsequence), and building each subsequence costs up to $O(n)$ to copy it into the result, giving

$$O(n \cdot 2^n).$$

Space Complexity. $O(n)$ for the recursion depth and the *current* list at any point in time (excluding the output storage, which is $O(n \cdot 2^n)$ to hold every generated subsequence).

Key Takeaway

The pick/not-pick recursion tree, with an explicit choose/recurse/un-choose rhythm, is the master template for this entire group of problems. Combination Sum, Subset Sum, and several other sections ahead are all this exact same tree, either pruned early (skip branches that can no longer succeed) or with the choice set widened beyond a simple binary pick/skip.

1.8 Check if There Exists a Subsequence with Sum K

Problem Statement. Given an array *arr* and an integer *k*, determine whether **any** subsequence of *arr* sums to exactly *k*.

Pattern Recognition

This is Section 1.7's pick/not-pick tree, but the question changes from “*generate every subsequence*” to “*does at least one satisfy a condition.*” Whenever a backtracking problem

only needs a yes/no answer (rather than every solution), the recursion should **stop exploring the instant an answer is found** – there is no need to generate the other $2^n - 1$ subsequences once one of them has already confirmed the answer is “yes.”

Understanding the problem carefully

Instead of building and recording a list at every leaf, track a running sum as the pick/not-pick decisions are made, and check it only once every element has been decided (at the base case). The function should return a boolean, and – critically – the moment either branch reports **true**, the calling level should return **true** immediately without bothering to explore the other branch at all.

Same Tree, but Short-Circuit on Success

EXISTS(*index*, *sum*): if *index* = *n*: return (*sum* = *k*). Otherwise: if EXISTS(*index* + 1, *sum* + *arr*[*index*]) (the pick branch) is **true**, return **true** immediately without checking the not-pick branch at all. Otherwise, return EXISTS(*index* + 1, *sum*) (the not-pick branch) directly as this call’s result.

Why short-circuiting never misses a valid answer

We only need to know whether *some* subsequence achieves sum *k* – not which one, nor how many do. The moment the pick branch reports success, we already have a definitive “yes” for the overall question, and continuing to explore the not-pick branch (or any other untried branch) could only ever confirm the same “yes” again, never change it to a “no.” Skipping that redundant exploration is therefore always safe, and in the best case it can save an enormous amount of work, since a successful subsequence might be found very early in the traversal while the vast majority of the 2^n possibilities are never even visited.

Algorithm

1. EXISTS(*index*, *sum*):
 - (a) If *index* = *n*: return (*sum* = *k*).
 - (b) If EXISTS(*index* + 1, *sum* + *arr*[*index*]): return **true**.
 - (c) Return EXISTS(*index* + 1, *sum*).
2. Call EXISTS(0, 0).

Dry run

Dry Run. Take *arr* = [1, 2, 3], *k* = 5.

- EXISTS(0, 0): pick 1: EXISTS(1, 1).
- EXISTS(1, 1): pick 2: EXISTS(2, 3).
- EXISTS(2, 3): pick 3: EXISTS(3, 6): base case, $6 \neq 5$, return **false**. Not-pick 3: EXISTS(3, 3): base case, $3 \neq 5$, return **false**. So EXISTS(2, 3) returns **false**.
- Back in EXISTS(1, 1): pick branch was **false**, try not-pick 2: EXISTS(2, 1): pick 3: EXISTS(3, 4): base, $4 \neq 5$, false. Not-pick 3: EXISTS(3, 1): base, $1 \neq 5$, false. So EXISTS(2, 1) is false, and EXISTS(1, 1) is false.

- Back in EXISTS(0,0): pick branch was **false**, try not-pick 1: EXISTS(1,0): pick 2: EXISTS(2,2): pick 3: EXISTS(3,5): base, $5 = 5!$ Return **true**. This propagates straight back up: EXISTS(2,2) returns **true** immediately (no need to check not-pick 3); EXISTS(1,0) returns **true** immediately; EXISTS(0,0) returns **true**.

The answer is **true**: the subsequence $\{2, 3\}$ sums to 5, and the recursion stopped exploring further branches the instant it was found.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool exists(int index, int sum, vector<int>& arr, int k) {
5     if (index == (int)arr.size()) return sum == k;
6
7     if (exists(index + 1, sum + arr[index], arr, k)) return true;
8     return exists(index + 1, sum, arr, k);
9 }
10
11 int main() {
12     vector<int> arr = {1, 2, 3};
13     cout << (exists(0, 0, arr, 5) ? "true" : "false") << endl;
14     return 0;
15 }
```

Complexity Analysis

Time Complexity. $O(2^n)$ in the worst case (no qualifying subsequence exists, so every branch must be explored), but often much faster in practice due to short-circuiting on the first success. **Space Complexity.** $O(n)$ for the recursion stack.

Key Takeaway

“Does there exist” questions should always short-circuit the moment a valid answer is found – propagate a **true** result immediately up through every level of the recursion, skipping any sibling branch that has not yet been explored. This is a small but important variation on the pick/not-pick template: the same tree shape, but with an early-exit discipline that the “generate everything” version of Section 1.7 does not need.

1.9 Count Subsequences with Sum K

Problem Statement. Given an array *arr* and an integer *k*, count **how many** subsequences of *arr* sum to exactly *k*.

Pattern Recognition

The third variation on Section 1.7’s pick/not-pick tree: from “*generate all*” (Section 1.7) to “*does one exist*” (Section 1.8) to, now, “*count how many*.” Counting needs neither full generation nor early short-circuiting – it needs both branches’ **counts** added together.

Understanding the problem carefully

Every complete pick/not-pick decision sequence produces exactly one subsequence, and the base case can directly check whether *that* subsequence's sum equals k , contributing 1 to the count if so, 0 otherwise. Since the pick and not-pick branches partition all subsequences into two disjoint groups (those that include $arr[index]$ and those that don't), the total count is simply the **sum** of the counts returned by both branches – no branch can be skipped, since both might contain qualifying subsequences.

Sum the Counts from Both Branches

COUNTSUM($index, sum$): if $index = n$: return 1 if $sum = k$, else 0. Otherwise: return COUNTSUM($index + 1, sum + arr[index]$) + COUNTSUM($index + 1, sum$).

Why summing (not short-circuiting) is required here

Unlike Section 1.8, where finding one success already fully answers the yes/no question, here a successful pick branch tells us nothing about how many *additional* successes might be hiding in the not-pick branch – both must be fully explored and their individual counts combined, since the two branches' qualifying subsequences are entirely different (disjoint) subsequences, both of which need to be counted toward the final total.

Algorithm

1. COUNTSUM($index, sum$):
 - (a) If $index = n$: return 1 if $sum = k$, else 0.
 - (b) Return COUNTSUM($index + 1, sum + arr[index]$) + COUNTSUM($index + 1, sum$).
2. Call COUNTSUM(0, 0).

Dry run

Dry Run. Take $arr = [1, 2, 1]$, $k = 2$.

- COUNTSUM(0, 0): pick 1: COUNTSUM(1, 1); not-pick: COUNTSUM(1, 0).
- COUNTSUM(1, 1) (came from picking index 0): pick 2: COUNTSUM(2, 3); not-pick: COUNTSUM(2, 1).
 - COUNTSUM(2, 3): pick 1: base, $sum = 4 \neq 2$, return 0. Not-pick: base, $sum = 3 \neq 2$, return 0. Total 0.
 - COUNTSUM(2, 1): pick 1: base, $sum = 2 = 2!$ return 1. Not-pick: base, $sum = 1 \neq 2$, return 0. Total 1.
- COUNTSUM(1, 1) = 0 + 1 = 1.
- COUNTSUM(1, 0) (came from not-picking index 0): pick 2: COUNTSUM(2, 2); not-pick: COUNTSUM(2, 0).
 - COUNTSUM(2, 2): pick 1: base, $sum = 3 \neq 2$, return 0. Not-pick: base, $sum = 2 = 2!$ return 1. Total 1.
 - COUNTSUM(2, 0): pick 1: base, $sum = 1 \neq 2$, return 0. Not-pick: base, $sum = 0 \neq 2$, return 0. Total 0.
- COUNTSUM(1, 0) = 1 + 0 = 1.

- $\text{COUNTSUM}(0, 0) = 1 + 1 = 2$.

The answer is 2: the qualifying subsequences are $\{1, 1\}$ (indices 0, 2) and $\{2\}$ (index 1 alone).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countSum(int index, int sum, vector<int>& arr, int k) {
5     if (index == (int)arr.size()) return sum == k ? 1 : 0;
6
7     int pick = countSum(index + 1, sum + arr[index], arr, k);
8     int notPick = countSum(index + 1, sum, arr, k);
9     return pick + notPick;
10 }
11
12 int main() {
13     vector<int> arr = {1, 2, 1};
14     cout << "Count: " << countSum(0, 0, arr, 2) << endl;
15     return 0;
16 }

```

Complexity Analysis

Time Complexity. $O(2^n)$ – every one of the 2^n subsequences must be considered, since any of them might contribute to the count. **Space Complexity.** $O(n)$ for the recursion stack.

Key Takeaway

The three variations seen so far on the same pick/not-pick tree – generate everything (Section 1.7), short-circuit on the first success (Section 1.8), and sum counts from both branches (this section) – are worth holding side by side as a single mental checklist: *generate, exists, or count* is usually the very first distinction to make about any new backtracking problem, since it determines how the two branches' results should be combined at every level.

1.10 Subset Sums I (Print All Subset Sums)

Problem Statement. Given an array *arr* of n integers, compute the **sum** of every one of its 2^n subsets, and return the list of all such sums (typically in sorted order).

Pattern Recognition

A direct simplification of Section 1.7's pick/not-pick tree: instead of recording the entire subsequence at each leaf, we only need to record its **sum** – a single number threaded through the recursion as a running total, rather than a list that must be built and torn down at every level.

Understanding the problem carefully

Exactly as in Section 1.7, every element independently contributes a pick/not-pick decision. The only change is what we choose to track: instead of appending the picked element to a growing list (and later removing it on backtrack), we simply add its value to a running sum parameter

(and, since this is passed by value rather than by reference, there is nothing to explicitly undo – each recursive call gets its own independent copy of the sum).

Thread a Running Sum Instead of a List

SOLVE(*index*, *sum*): if *index* = *n*: record *sum* as one subset's total; return. Otherwise: SOLVE(*index* + 1, *sum* + *arr*[*index*]) (pick), then SOLVE(*index* + 1, *sum*) (not-pick).

Why no explicit undo step is needed here

In Section 1.7, *current* was a single shared, mutated list, so the pick branch's append had to be explicitly undone before exploring the not-pick branch. Here, *sum* is passed **by value** (a plain number, copied into each recursive call rather than referenced and mutated in place), so the pick branch's modified sum simply ceases to exist once that branch's call returns – the not-pick branch automatically continues with the original, unmodified sum from this level, with no explicit “undo” arithmetic required. This is a useful general lesson: whether backtracking needs an explicit undo step depends entirely on whether the state being threaded through is mutated in place (needing undo) or passed by value (automatically restored).

Algorithm

1. SOLVE(*index*, *sum*):
 - (a) If *index* = *n*: append *sum* to the result list; return.
 - (b) SOLVE(*index* + 1, *sum* + *arr*[*index*]).
 - (c) SOLVE(*index* + 1, *sum*).
2. Call SOLVE(0, 0); sort the result list before returning (if sorted output is required).

Dry run

Dry Run. Take *arr* = [2, 3].

- SOLVE(0, 0): pick 2: SOLVE(1, 2).
 - Pick 3: SOLVE(2, 5): base, record 5.
 - Not-pick 3: SOLVE(2, 2): base, record 2.
- Not-pick 2: SOLVE(1, 0).
 - Pick 3: SOLVE(2, 3): base, record 3.
 - Not-pick 3: SOLVE(2, 0): base, record 0.

Recorded sums: 5, 2, 3, 0; sorted: 0, 2, 3, 5 – matching the four subsets {}, {2}, {3}, {2, 3}.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(int index, int sum, vector<int>& arr, vector<int>& result) {
5     if (index == (int)arr.size()) {
6         result.push_back(sum);
7         return;

```

```

8     }
9     solve(index + 1, sum + arr[index], arr, result);
10    solve(index + 1, sum, arr, result);
11 }
12
13 vector<int> subsetSums(vector<int>& arr) {
14     vector<int> result;
15     solve(0, 0, arr, result);
16     sort(result.begin(), result.end());
17     return result;
18 }
19
20 int main() {
21     vector<int> arr = {2, 3};
22     for (int s : subsetSums(arr)) cout << s << " ";
23     cout << endl;
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. $O(2^n)$ to generate all sums, plus $O(2^n \log(2^n)) = O(n \cdot 2^n)$ if sorting the output is required. **Space Complexity.** $O(n)$ for the recursion stack, plus $O(2^n)$ for the output.

Key Takeaway

Whether a backtracking branch needs an explicit “undo” depends on whether its state is mutated in place (a shared list, needing an undo) or passed by value (a plain number or immutable structure, automatically restored when the branch returns). Recognizing which kind of state a problem needs is often the deciding factor in how cleanly the recursion can be written.

1.11 Subset Sums II (Unique Subsets with Duplicates Present)

Problem Statement. Given an array *arr* of *n* integers that **may contain duplicate values**, return every distinct subset (the power set), with **no duplicate subsets** in the output, even though duplicate elements exist in the input.

Pattern Recognition

The keyword “*may contain duplicates*,” combined with “*no duplicate subsets in the output*,” is the trigger for this chapter’s other major backtracking template, distinct from pick/not-pick: **sort first, then iterate through choices at each recursive level, explicitly skipping any choice equal to the immediately previous choice already tried at the same level.** This “sort, then skip same-level duplicates” technique is reused in every later problem that needs to handle duplicate input values (Sections 1.13–1.14).

Understanding the problem carefully

If the input has duplicate values, the plain pick/not-pick tree from Section 1.7 would generate the *same* subset multiple times – for example, with *arr* = [1, 2, 2], picking “the first 2” and picking “the second 2” (while skipping the other) produce two different leaves in the recursion tree, but the exact same subset {1, 2}. To avoid this, we change the recursion’s shape: instead

of a binary pick/not-pick at every fixed index, we **sort** the array first (so equal values become adjacent), and at each recursive level, we loop through the remaining choices, but skip any choice that is equal to the choice immediately before it *at this same level* (not across different levels, where repeats are exactly what we want, to build subsets containing multiple copies of a repeated value).

Sort First, Then Skip Same-Level Duplicates

Sort *arr*. SOLVE(*start*, *current*): record a copy of *current* as one valid subset immediately (every prefix explored is itself a valid, distinct subset, not just the leaves). For *i* from *start* to *n* - 1: if *i* > *start* and *arr*[*i*] = *arr*[*i* - 1], skip this *i* (this exact value has already been tried as the *first* new element added at this level, so trying it again here would only reproduce a subset already generated). Otherwise: append *arr*[*i*] to *current*; recurse SOLVE(*i* + 1, *current*); remove *arr*[*i*] from *current* (undo).

Why skipping only at the same level (not across levels) is correct

The key distinction: *arr*[*i*] = *arr*[*i* - 1] being skipped only applies when both are being considered as candidates to fill the same “next new element” slot within the same call to SOLVE – i.e. within the same **for** loop, at the same recursion depth. A repeated value is never forbidden from appearing *multiple times within one subset* (e.g. {2, 2} from *arr* = [1, 2, 2] is a perfectly valid, distinct subset) – what is forbidden is generating the *same subset twice* by choosing “the first 2 at this position” in one branch and “the second 2 at this same position” in a sibling branch, when both choices would lead to identical subsequent subsets. Since the array is sorted, any two equal values are guaranteed to be adjacent in the loop, so checking *arr*[*i*] = *arr*[*i* - 1] exactly identifies this situation.

Algorithm

1. Sort *arr*.
2. SOLVE(*start*, *current*):
 - (a) Append a copy of *current* to the result.
 - (b) For *i* from *start* to *n* - 1:
 - If *i* > *start* and *arr*[*i*] = *arr*[*i* - 1]: skip (continue to next *i*).
 - Append *arr*[*i*] to *current*; recurse SOLVE(*i* + 1, *current*); remove *arr*[*i*] (undo).
3. Call SOLVE(0, []).

Dry run

Dry Run. Take *arr* = [1, 2, 2] (already sorted).

- SOLVE(0, []): record []. Loop *i* = 0 (1): append. SOLVE(1, [1]). Undo.
 - SOLVE(1, [1]): record [1]. Loop *i* = 1 (2): append. SOLVE(2, [1, 2]). Undo. *i* = 2 (2): *i* > *start* = 1 and *arr*[2] = *arr*[1] = 2: skip.
 - * SOLVE(2, [1, 2]): record [1, 2]. Loop *i* = 2 (2): append. SOLVE(3, [1, 2, 2]): record [1, 2, 2]. Undo.
- Loop continues at top level: *i* = 1 (2): *i* > *start* = 0, but *arr*[1] ≠ *arr*[0] (2 ≠ 1), no skip. Append. SOLVE(2, [2]). Undo.

– SOLVE(2, [2]): record [2]. Loop $i = 2$ (2): append. SOLVE(3, [2, 2]): record [2, 2]. Undo.

- $i = 2$ at top level: $i > start = 0$ and $arr[2] = arr[1] = 2$: skip.

Recorded subsets: $[], [1], [1, 2], [1, 2, 2], [2], [2, 2]$ – exactly the 6 distinct subsets of $\{1, 2, 2\}$ (note there are 6, not $2^3 = 8$, because $\{2\}$ and $\{2, 2\}$ each appear in only one form despite the duplicate 2).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(int start, vector<int>& arr, vector<int>& current,
5           vector<vector<int>>& result) {
6     result.push_back(current);
7
8     for (int i = start; i < (int)arr.size(); i++) {
9         if (i > start && arr[i] == arr[i - 1]) continue; // skip
10                duplicate
11
12         current.push_back(arr[i]);
13         solve(i + 1, arr, current, result);
14         current.pop_back();
15     }
16 }
17
18 vector<vector<int>> subsetsWithDup(vector<int>& arr) {
19     sort(arr.begin(), arr.end());
20     vector<vector<int>> result;
21     vector<int> current;
22     solve(0, arr, current, result);
23     return result;
24 }
25
26 int main() {
27     vector<int> arr = {1, 2, 2};
28     for (auto& subset : subsetsWithDup(arr)) {
29         cout << "[ ";
30         for (int x : subset) cout << x << " ";
31         cout << "] ";
32     }
33     cout << endl;
34     return 0;
35 }
```

Complexity Analysis

Time Complexity. Sorting costs $O(n \log n)$. The recursion visits each distinct subset once, and building/copying each costs up to $O(n)$, giving an overall time complexity bounded by

$$O(n \cdot 2^n)$$

in the worst case (no duplicates, so all 2^n subsets are distinct). **Space Complexity.** $O(n)$ for the recursion depth and *current* list.

Key Takeaway

“Sort first, then skip a choice equal to the immediately previous choice tried at the same recursion level” is the standard fix for duplicate-avoidance in backtracking, and it generalizes far beyond subsets: the same technique, applied to the choice loop inside Combination Sum II (Section 1.13) and Combination Sum III (Section 1.14), prevents duplicate combinations there too.

1.12 Combination Sum

Problem Statement. Given an array *candidates* of **distinct** positive integers and a target *target*, return every unique combination of candidates that sums to *target*. The **same candidate may be used an unlimited number of times** within one combination.

Pattern Recognition

“The same element may be reused unlimited times” is the keyword that distinguishes this from Section 1.7’s plain pick/not-pick tree: the **pick** branch must **stay at the same index** (since this element remains available for reuse) rather than advancing to the next index, while the **not-pick** branch still advances (moving permanently past this candidate, never to reconsider it).

Understanding the problem carefully

At each index, we still face a binary choice – use this candidate (and possibly use it again later) or move past it forever – but “use this candidate” no longer means “use it exactly once and move on”: because reuse is unlimited, choosing to include *candidates[index]* should recurse back into the *same* index, leaving open the possibility of including it again. The recursion only definitively moves past an index once the not-pick branch is taken.

Pick Stays, Not-Pick Advances

SOLVE(*index*, *remaining*, *current*): if *remaining* = 0: record *current*; return. If *index* = *n* or *remaining* < 0: return (failed branch, either ran out of candidates or overshoot the target). Otherwise: append *candidates[index]* to *current*; recurse SOLVE(*index*, *remaining* – *candidates[index]*, *current*) (pick, staying at the same index); remove the appended element (undo); recurse SOLVE(*index* + 1, *remaining*, *current*) (not-pick, advancing).

Why staying at the same index correctly models unlimited reuse

If the pick branch advanced to *index* + 1 as in Section 1.7, each candidate could be used at most once per combination – exactly the wrong behavior for this problem. Staying at *index* after picking means the very next recursive call still has *candidates[index]* available to pick *again*, and can keep doing so for as many repetitions as the remaining target allows, only ever moving past *index* once the not-pick branch is explicitly taken (which represents “permanently done with this candidate, for this branch of the search”). The *remaining* < 0 check ensures the recursion does not loop forever picking a positive candidate repeatedly past the point where it could still sum to the target – it is the base-case escape that makes termination certain despite the same index being revisited arbitrarily many times.

Algorithm

1. SOLVE(*index*, *remaining*, *current*):

- (a) If *remaining* = 0: record *current*; return.
- (b) If *index* = *n* or *remaining* < 0: return.
- (c) Append *candidates[index]*; SOLVE(*index*, *remaining* − *candidates[index]*, *current*); remove (undo).
- (d) SOLVE(*index* + 1, *remaining*, *current*).

2. Call SOLVE(0, *target*, []).

Dry run

Dry Run. Take *candidates* = [2, 3], *target* = 5.

- SOLVE(0, 5, []): pick 2: SOLVE(0, 3, [2]) (stays at index 0).
 - Pick 2 again: SOLVE(0, 1, [2, 2]). Pick 2 again: SOLVE(0, −1, [2, 2, 2]): *remaining* < 0, return (fail). Undo. Not-pick: SOLVE(1, 1, [2, 2]): pick 3: SOLVE(1, −2, ...): fail. Undo. Not-pick: SOLVE(2, 1, [2, 2]): *index* = *n* = 2, return (fail).
 - Undo back to [2]. Not-pick 3 at this level: SOLVE(1, 3, [2]): pick 3: SOLVE(1, 0, [2, 3]): *remaining* = 0! Record [2, 3].
- Back at top: undo to []. Not-pick 2: SOLVE(1, 5, []): pick 3: SOLVE(1, 2, [3]): pick 3 again: SOLVE(1, −1, [3, 3]): fail. Not-pick: SOLVE(2, 2, [3]): *index* = *n*, fail.

The only recorded combination is [2, 3]. (Note: in a full trace, [2, 2, ...] paths never reach exactly 0 remaining and so never record anything, which the dry run above confirms.)

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void solve(int index, int remaining, vector<int>& candidates,
5            vector<int>& current, vector<vector<int>>& result) {
6      if (remaining == 0) {
7          result.push_back(current);
8          return;
9      }
10     if (index == (int)candidates.size() || remaining < 0) return;
11
12     current.push_back(candidates[index]);
13     solve(index, remaining - candidates[index], candidates, current,
14           result);
15     current.pop_back();
16
17     solve(index + 1, remaining, candidates, current, result);
18 }
19
20 vector<vector<int>> combinationSum(vector<int>& candidates, int target)
21 {
22     vector<vector<int>> result;
23     vector<int> current;
24     solve(0, target, candidates, current, result);
25     return result;
26 }
```

```

26 int main() {
27     vector<int> candidates = {2, 3, 6, 7};
28     for (auto& comb : combinationSum(candidates, 7)) {
29         cout << "[ ";
30         for (int x : comb) cout << x << " ";
31         cout << "] ";
32     }
33     cout << endl;
34     return 0;
35 }

```

Complexity Analysis

Time Complexity. Difficult to bound tightly in general (it depends on the target and candidate values), but is exponential in the worst case, roughly $O(2^{\text{target}})$ for the branching factor at each remaining-target unit; in practice it is bounded by the number of valid combinations times their average length. **Space Complexity.** $O(\text{target} / \min(\text{candidates}))$ for the maximum recursion depth (the most times the smallest candidate could be picked before exceeding the target).

Key Takeaway

“Unlimited reuse of the same element” converts the pick branch’s target index from “advance to *index* + 1” to “stay at *index*.” This single change is the entire difference between Combination Sum (this section) and the no-reuse subset/combination problems before and after it – worth recognizing on sight whenever a problem statement explicitly allows repeated use of the same value.

1.13 Combination Sum II

Problem Statement. Given an array *candidates* that **may contain duplicate values** and a target, return every unique combination that sums to the target, where **each individual element of the array may be used at most once** (no reuse), and the output must contain no duplicate combinations.

Pattern Recognition

This problem directly combines two techniques already derived in this chapter: Section 1.11’s “sort, then skip a choice equal to the previous choice at the same level” duplicate-avoidance, and Section 1.12’s general combination-building recursion – but **without** that section’s “stay at the same index” reuse rule, since each element here may only be used once.

Understanding the problem carefully

Because reuse is forbidden, the recursion always advances past the current index after picking it (exactly like Section 1.11, never like Section 1.12). Because duplicate values are present, we sort first and skip same-level duplicate choices, exactly as in Section 1.11, to avoid generating the same combination more than once. One further optimization becomes available specifically because we additionally have a numeric target and a sorted array: the moment a candidate’s value alone exceeds the remaining target, every later candidate (being at least as large, since the array is sorted) must also exceed it, so the entire rest of the loop at this level can be abandoned immediately, rather than checked one by one.

Sort, Skip Same-Level Duplicates, and Break Early

Sort *candidates*. SOLVE(*start*, *remaining*, *current*): if *remaining* = 0: record *current*; return. For *i* from *start* to *n* - 1: if *i* > *start* and *candidates*[*i*] = *candidates*[*i* - 1]: skip (same-level duplicate). If *candidates*[*i*] > *remaining*: **break** out of the loop entirely (pruning – no further *i* in this sorted loop can help). Otherwise: append *candidates*[*i*]; recurse SOLVE(*i* + 1, *remaining* - *candidates*[*i*], *current*); remove (undo).

Why breaking (not just skipping) on an over-large candidate is valid

This is a direct consequence of sortedness, not a heuristic: since *candidates* is sorted in increasing order, once *candidates*[*i*] > *remaining*, every subsequent *candidates*[*j*] for *j* > *i* satisfies *candidates*[*j*] ≥ *candidates*[*i*] > *remaining* too – so none of them could possibly lead to a valid combination either, at this level. Breaking the loop entirely (rather than merely skipping this one *i* and checking the next) is therefore a safe pruning step that discards an entire suffix of guaranteed-failing branches in $O(1)$, rather than wastefully visiting and individually rejecting each one.

Algorithm

1. Sort *candidates*.
2. SOLVE(*start*, *remaining*, *current*):
 - (a) If *remaining* = 0: record *current*; return.
 - (b) For *i* from *start* to *n* - 1:
 - If *i* > *start* and *candidates*[*i*] = *candidates*[*i* - 1]: continue (skip).
 - If *candidates*[*i*] > *remaining*: break.
 - Append *candidates*[*i*]; recurse SOLVE(*i* + 1, *remaining* - *candidates*[*i*], *current*); remove (undo).
3. Call SOLVE(0, *target*, []).

Dry run

Dry Run. Take *candidates* = [1, 1, 2] (sorted), *target* = 3.

- SOLVE(0, 3, []): *i* = 0 (1): append, recurse SOLVE(1, 2, [1]). Undo.
 - SOLVE(1, 2, [1]): *i* = 1 (1): *i* = *start*, no skip check needed. Append, recurse SOLVE(2, 1, [1, 1]). Undo.
 - * SOLVE(2, 1, [1, 1]): *i* = 2 (2): 2 > 1, **break** immediately.
 - *i* = 2 (2) at this level: append, recurse SOLVE(3, 0, [1, 2]): *remaining* = 0! Record [1, 2].
- Back at top level: *i* = 1 (1): *i* > *start* = 0 and *candidates*[1] = *candidates*[0]: skip.
- *i* = 2 (2): append, recurse SOLVE(3, 1, [2]): *remaining* = 1 ≠ 0, and the loop from *i* = 3 to *n* - 1 is empty (no candidates left); this branch records nothing.

Recorded combinations: [1, 1, 1]? – wait, recheck: we recorded [1, 2] once. The full correct output for this classic instance is {[1, 1, 1]-like attempts that failed, [1, 2]}; tracing carefully as above, the only successful combination found is [1, 2], matching the expected unique result for target 3 from {1, 1, 2} (the combination 1 + 1 + 2 = 4 ≠ 3 does not qualify; 1 + 2 = 3 does, exactly once despite two copies of 1 being available).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(int start, int remaining, vector<int>& candidates,
5           vector<int>& current, vector<vector<int>>& result) {
6     if (remaining == 0) {
7         result.push_back(current);
8         return;
9     }
10    for (int i = start; i < (int)candidates.size(); i++) {
11        if (i > start && candidates[i] == candidates[i - 1]) continue;
12        if (candidates[i] > remaining) break;
13
14        current.push_back(candidates[i]);
15        solve(i + 1, remaining - candidates[i], candidates, current,
16              result);
17        current.pop_back();
18    }
19 }
20
21 vector<vector<int>> combinationSum2(vector<int>& candidates, int target
22 ) {
23     sort(candidates.begin(), candidates.end());
24     vector<vector<int>> result;
25     vector<int> current;
26     solve(0, target, candidates, current, result);
27     return result;
28 }
29
30 int main() {
31     vector<int> candidates = {1, 1, 2};
32     for (auto& comb : combinationSum2(candidates, 3)) {
33         cout << "[ ";
34         for (int x : comb) cout << x << " ";
35         cout << "] ";
36     }
37     cout << endl;
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(2^n)$ in the worst case (every subset potentially checked), though sorting plus the early-break pruning typically eliminates a substantial fraction of branches in practice; an additional $O(n \log n)$ for the initial sort. **Space Complexity.** $O(n)$ for the recursion depth.

Key Takeaway

Combination Sum II is a direct composition of two already-derived techniques – no-reuse advancing recursion plus sort-and-skip duplicate avoidance – with one additional sorted-array pruning trick (breaking early once a candidate exceeds the remaining target) layered on top. Recognizing a new problem as “technique A combined with technique B” is, again, the fastest path to a correct solution.

1.14 Combination Sum III

Problem Statement. Find all valid combinations of exactly k **distinct** numbers, chosen from 1 through 9 (each usable at most once across all combinations, i.e. no repeats within one combination), that sum to exactly n .

Pattern Recognition

A further specialization of Section 1.13's no-reuse, advancing recursion: the candidate pool is now the fixed, already-sorted range 1 through 9 (so no separate sort step or duplicate-skip is needed, since this fixed pool has no duplicates by construction), and one new constraint is added – the combination's **length** must be exactly k , not just its sum.

Understanding the problem carefully

The recursion is the same advancing, no-reuse shape as Combination Sum II, but simplified by the fixed candidate pool $\{1, \dots, 9\}$ (already sorted, no duplicates to worry about), and with the base case now checking *two* conditions instead of one: both the running count of chosen numbers must equal k , and the running remaining target must have reached exactly 0, at the same time.

Same Template, with a Length Constraint Added to the Base Case

SOLVE(*start*, *remaining*, *current*): if $|current| = k$: record *current* if *remaining* = 0 (otherwise this length- k combination simply doesn't sum correctly, and we discard it); return either way (no further recursion needed once length k is reached). Otherwise, for i from *start* to 9: if $i > remaining$: break (the same sorted-range pruning as Combination Sum II). Append i ; recurse SOLVE($i + 1$, $remaining - i$, *current*); remove (undo).

Why checking both conditions together at length k is necessary

A combination might reach exactly k chosen numbers without summing to n (too small or too large), or might sum to exactly n using fewer or more than k numbers – neither partial success is acceptable, since the problem requires *both* conditions simultaneously. Stopping the recursion the instant the count reaches k (regardless of whether the sum is currently correct) avoids wasted exploration beyond the required length, while still checking the sum condition explicitly before recording, since reaching length k alone does not guarantee the target sum has also been met.

Algorithm

1. SOLVE(*start*, *remaining*, *current*):
 - (a) If $|current| = k$: if *remaining* = 0: record *current*; return.
 - (b) For i from *start* to 9:
 - If $i > remaining$: break.
 - Append i ; recurse SOLVE($i + 1$, $remaining - i$, *current*); remove (undo).
2. Call SOLVE(1, n , []).

Dry run

Dry Run. Take $k = 3$, $n = 7$.

- SOLVE(1, 7, []): $i = 1$: append, recurse SOLVE(2, 6, [1]).
 - $i = 2$: append, recurse SOLVE(3, 4, [1, 2]).
 - * $|current| = 2 \neq 3$ yet. $i = 3$: $3 \leq 4$, append, recurse SOLVE(4, 1, [1, 2, 3]): $|current| = 3 = k$, but $remaining = 1 \neq 0$: discard. $i = 4$: $4 > 1$, break.
 - Undo. $i = 3$ (at [1] level): append, recurse SOLVE(4, 3, [1, 3]): $i = 4$: $4 > 3$, break (no length-3 combination found here).
- Continue similarly... eventually reaching $i = 1, i = 2, i = 4$: SOLVE(2, 6, [1]) $\rightarrow i = 4$: $4 \leq 6$, append, recurse SOLVE(5, 2, [1, 4]): $i = 5$: $5 > 2$, break.
- Trying $i = 1, i = 6$ at the [1] level... continuing the full search eventually finds [1, 2, 4] ($1 + 2 + 4 = 7$, length 3) as a valid recorded combination.

(The complete search also finds no other length-3 combinations from $\{1, \dots, 9\}$ summing to 7 besides [1, 2, 4], which the C++ code below confirms by direct execution.)

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void solve(int start, int remaining, int k, vector<int>& current,
5            vector<vector<int>>& result) {
6      if ((int)current.size() == k) {
7          if (remaining == 0) result.push_back(current);
8          return;
9      }
10     for (int i = start; i <= 9; i++) {
11         if (i > remaining) break;
12
13         current.push_back(i);
14         solve(i + 1, remaining - i, k, current, result);
15         current.pop_back();
16     }
17 }
18
19 vector<vector<int>> combinationSum3(int k, int n) {
20     vector<vector<int>> result;
21     vector<int> current;
22     solve(1, n, k, current, result);
23     return result;
24 }
25
26 int main() {
27     for (auto& comb : combinationSum3(3, 7)) {
28         cout << "[ ";
29         for (int x : comb) cout << x << " ";
30         cout << "] ";
31     }
32     cout << endl;
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. The candidate pool is fixed at 9 values, so the search space is bounded by $\binom{9}{k}$, a constant independent of any input size – effectively $O(1)$ for this fixed-range problem, though conventionally still described as exponential in the (here, fixed) pool size.

Space Complexity. $O(k)$ for the recursion depth.

Key Takeaway

A fixed, small, already-sorted, duplicate-free candidate pool (like 1–9 here) simplifies a backtracking template by removing the need for an initial sort or a same-level duplicate-skip check – but the core advancing, no-reuse recursion, plus sorted-pool pruning on the remaining target, is unchanged from Combination Sum II. Adding a second base-case condition (here, an exact length k) is a small, composable extension worth recognizing as its own reusable idea.

1.15 Generate Parentheses

Problem Statement. Given an integer n , generate all combinations of **well-formed** (valid) parenthesis strings using exactly n pairs of parentheses.

Pattern Recognition

“Generate all valid ...” signals backtracking with **validity-aware pruning**: rather than generating all $\binom{2n}{n}$ arrangements of n open and n close parentheses and filtering out the invalid ones afterward (as in the Stack and Queue chapter’s Valid Parentheses problem *checking* a given string), we instead only ever make a choice that keeps the string-so-far on track to *possibly* become valid, pruning illegal branches the moment they would become unsalvageable.

Understanding the problem carefully

At any point while building the string, two counts matter: how many open parentheses have been placed so far, and how many close parentheses have been placed so far. An open parenthesis can be placed as long as fewer than n have been used so far (we must not exceed n total). A close parenthesis can be placed only if doing so would not create more closes than opens at any prefix – i.e. only if the current count of closes is still strictly less than the current count of opens (otherwise an unmatched close would appear, which the Stack and Queue chapter’s Valid Parentheses problem already established invalidates a parenthesis string).

Two Guarded Choices Instead of a Binary Pick/Not-Pick

SOLVE(*current*, *openCount*, *closeCount*): if $|current| = 2n$: record *current*; return. Otherwise: if *openCount* < n : append '('; recurse with *openCount* + 1; remove (undo). If *closeCount* < *openCount*: append ')'; recurse with *closeCount* + 1; remove (undo).

Why these two guards are exactly the validity conditions, applied early

This is the backtracking analogue of the Stack and Queue chapter’s Valid Parentheses stack-based validity check, applied *during construction* rather than after the fact: a parenthesis string is valid exactly when every prefix has at least as many opens as closes, and the total opens and closes both equal n . The guard *closeCount* < *openCount* is precisely “this prefix currently has

more opens than closes, so one more close is still safe”; the guard $openCount < n$ simply caps the total opens at n . Because these guards are checked *before* a character is ever placed, no invalid prefix is ever constructed in the first place – the search space explored is exactly the valid strings (and their valid-so-far prefixes), never the much larger space of all arrangements.

Algorithm

1. SOLVE(*current*, *openCount*, *closeCount*):
 - (a) If $|current| = 2n$: record *current*; return.
 - (b) If $openCount < n$: append '('; recurse; remove.
 - (c) If $closeCount < openCount$: append ')'; recurse; remove.
2. Call SOLVE("", 0, 0).

Dry run

Dry Run. Take $n = 2$.

- SOLVE("", 0, 0): open allowed ($0 < 2$): append (. SOLVE("(", 1, 0).
 - Open allowed ($1 < 2$): append (. SOLVE("(", 2, 0).
 - * Open not allowed ($2 \not< 2$). Close allowed ($0 < 2$): append). SOLVE("()", 2, 1): close allowed ($1 < 2$): append). SOLVE("()", 2, 2): length $4 = 2n$, record "()".
 - Undo back to "(". Close allowed ($0 < 1$): append). SOLVE("()", 1, 1): open allowed ($1 < 2$): append (. SOLVE("()(", 2, 1): close allowed ($1 < 2$): append). SOLVE("()()", 2, 2): record "()()".

Recorded strings: "()", "()()" – exactly the 2 valid parenthesizations of $n = 2$ pairs.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void solve(string& current, int openCount, int closeCount, int n,
5            vector<string>& result) {
6      if ((int)current.size() == 2 * n) {
7          result.push_back(current);
8          return;
9      }
10     if (openCount < n) {
11         current.push_back('(');
12         solve(current, openCount + 1, closeCount, n, result);
13         current.pop_back();
14     }
15     if (closeCount < openCount) {
16         current.push_back(')');
17         solve(current, openCount, closeCount + 1, n, result);
18         current.pop_back();
19     }
20 }
21
22 vector<string> generateParenthesis(int n) {
23     vector<string> result;
```

```

24     string current;
25     solve(current, 0, 0, n, result);
26     return result;
27 }
28
29 int main() {
30     for (string& s : generateParenthesis(3)) cout << s << " ";
31     cout << endl;
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. The number of valid strings generated is the n -th Catalan number, $C_n = \binom{2n}{n}/(n+1)$, and each costs $O(n)$ to build, giving an overall time complexity of

$$O\left(\frac{4^n}{\sqrt{n}}\right)$$

(the asymptotic growth rate of the Catalan numbers), substantially smaller than the $O(2^{2n})$ that generating and filtering all arrangements would cost. **Space Complexity.** $O(n)$ for the recursion depth (the string grows to length $2n$, but the call depth is also $O(n)$ since two characters can be added back to back in the worst case, or more precisely the depth equals the string length built so far, $O(n)$ at the deepest).

Key Takeaway

Whenever a backtracking problem's notion of "validity" can be checked *incrementally* (true/false after every partial choice, not only at the very end), build that incremental check directly into the recursion as a guard before each choice, rather than generating complete candidates and validating them afterward. This is dramatically more efficient whenever invalid prefixes are common, since entire invalid subtrees are never explored at all.

1.16 Generate All Binary Strings with No Two Consecutive 1s

Problem Statement. Given an integer n , generate all binary strings of length n such that no two 1s are adjacent (no two consecutive positions both hold 1).

Pattern Recognition

Another instance of Section 1.15's incremental-validity-guard pattern: instead of generating all 2^n binary strings and filtering, build the constraint directly into the choice of what character can be placed **next**, based only on **the single most recently placed character** – a guard that depends on a small, fixed amount of local history rather than the entire string built so far.

Understanding the problem carefully

At every position, placing a 0 is always safe, regardless of what came before – it can never create two consecutive 1s. Placing a 1 is safe *only if* the immediately preceding character (if any) was 0 – placing it right after another 1 would violate the constraint. So the only piece of history the recursion needs to remember is a single boolean: "was the last character placed a 1?"

Guard the Choice of 1 by the Last Character Placed

SOLVE(*current*, *lastWasOne*): if $|current| = n$: record *current*; return. Always: append 0; recurse with *lastWasOne* = false; remove (undo). If *lastWasOne* is false (i.e. the previous character was not 1, or this is the first character): append 1; recurse with *lastWasOne* = true; remove (undo).

Why remembering only the single last character is sufficient

The constraint “no two consecutive 1s” is a purely **local** property – it only ever compares two *adjacent* positions, never anything further apart. So the only information needed to decide whether placing a 1 right now is safe is whether the immediately preceding position holds a 1; nothing about positions further back can affect this decision. This is what allows the recursion to carry just one bit of state (rather than re-scanning the entire *current* string at every step to check for violations), keeping each step’s validity check $O(1)$.

Algorithm

1. SOLVE(*current*, *lastWasOne*):
 - (a) If $|current| = n$: record *current*; return.
 - (b) Append 0; recurse with *lastWasOne* = false; remove (undo).
 - (c) If not *lastWasOne*: append 1; recurse with *lastWasOne* = true; remove (undo).
2. Call SOLVE("", false).

Dry run

Dry Run. Take $n = 3$.

- SOLVE("", false): place 0: SOLVE("0", false).
 - Place 0: SOLVE("00", false): place 0: record "000"; place 1 (allowed, last was 0): record "001".
 - Place 1 (allowed): SOLVE("01", true): place 0: record "010"; place 1 – **not allowed** (last was 1), skipped entirely.
- Place 1 (allowed at the very first position): SOLVE("1", true): place 0: SOLVE("10", false): place 0: record "100"; place 1 (allowed): record "101". Place 1 – not allowed (last was 1), skipped.

Recorded strings: "000", "001", "010", "100", "101" – exactly the 5 length-3 binary strings with no two consecutive 1s (matching the Fibonacci-like count $F(n+2)$ of such strings).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(string& current, bool lastWasOne, int n, vector<string>&
   result) {
5     if ((int)current.size() == n) {
6         result.push_back(current);
7         return;

```

```

8     }
9     current.push_back('0');
10    solve(current, false, n, result);
11    current.pop_back();
12
13    if (!lastWasOne) {
14        current.push_back('1');
15        solve(current, true, n, result);
16        current.pop_back();
17    }
18 }
19
20 vector<string> generateBinaryStrings(int n) {
21     vector<string> result;
22     string current;
23     solve(current, false, n, result);
24     return result;
25 }
26
27 int main() {
28     for (string& s : generateBinaryStrings(3)) cout << s << " ";
29     cout << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. The number of valid strings of length n is $O(\phi^n)$ (a Fibonacci-like growth rate, where ϕ is the golden ratio), each costing $O(n)$ to build, giving an overall time complexity of

$$O(n \cdot \phi^n),$$

substantially smaller than the $O(n \cdot 2^n)$ of generating and filtering all binary strings. **Space Complexity.** $O(n)$ for the recursion depth.

Key Takeaway

When a backtracking constraint only ever depends on a small, fixed amount of recent history (here, just the single previous character), thread exactly that small amount of state through the recursion's parameters, rather than re-deriving it by re-scanning the entire candidate built so far. This keeps each step's validity check $O(1)$ and generalizes to any constraint expressible in terms of a bounded “sliding window” of recent choices.

1.17 Letter Combinations of a Phone Number

Problem Statement. Given a string of digits from 2 to 9, where each digit maps to a fixed set of letters exactly as on a telephone keypad (e.g. $2 \rightarrow \text{abc}$, $3 \rightarrow \text{def}$, ..., $7 \rightarrow \text{pqrs}$, $9 \rightarrow \text{wxyz}$), return every possible letter combination obtainable by choosing one letter for each digit, in order.

Pattern Recognition

“Choose one option per position, from a different choice set at each position” generalizes Section 1.7’s binary pick/not-pick into an n -ary branch at every level: instead of 2 choices (include/exclude) at each position, there are however many letters the current digit maps to,

and the recursion simply loops over all of them instead of hard-coding exactly two branches.

Understanding the problem carefully

Each digit position contributes one letter to the final combination, chosen independently from that digit's fixed mapping (e.g. digit '2' contributes one of a, b, c). Since these choices are independent across positions, exactly as the pick/not-pick choices were independent across array elements in Section 1.7, the total number of combinations is the **product** of each digit's letter count, and a recursion that loops through all letters for the current digit, recursing once per letter, visits every combination exactly once.

Loop Over the Current Digit's Letters Instead of a Fixed Binary Choice

SOLVE(*index*, *current*): if *index* = |*digits*|: record *current*; return. Otherwise: for each letter *c* in the mapping for *digits*[*index*]: append *c* to *current*; recurse SOLVE(*index* + 1, *current*); remove *c* (undo).

Why looping generalizes the binary pick/not-pick template directly

The pick/not-pick recursion in Section 1.7 is the special case of this loop where every position's choice set happens to have exactly 2 options ("include" or "exclude"). Here, the choice set size varies by digit (e.g. 3 letters for most digits, but 4 for 7 and 9), so the recursion is written as a general loop over however many choices exist at the current position, rather than two separately hard-coded branches – the same choose/recurse/un-choose rhythm applies uniformly, regardless of how many choices the loop iterates over.

Algorithm

1. Define the digit-to-letters mapping (a fixed lookup table).
2. SOLVE(*index*, *current*):
 - (a) If *index* = |*digits*|: record *current*; return.
 - (b) For each letter *c* mapped from *digits*[*index*]: append *c*; recurse SOLVE(*index* + 1, *current*); remove (undo).
3. Call SOLVE(0, "") (return an empty list immediately if *digits* is empty, as a special case).

Dry run

Dry Run. Take *digits* = "23" (2 → abc, 3 → def).

- SOLVE(0, ""): loop over a, b, c for digit '2'.
 - *c* = a: SOLVE(1, "a"): loop over d, e, f for digit '3': record "ad", "ae", "af".
 - *c* = b: SOLVE(1, "b"): record "bd", "be", "bf".
 - *c* = c: SOLVE(1, "c"): record "cd", "ce", "cf".

Result: "ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf" – all $3 \times 3 = 9$ combinations.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(int index, string& digits, string& current,
5           vector<string>& mapping, vector<string>& result) {
6     if (index == (int)digits.size()) {
7         result.push_back(current);
8         return;
9     }
10    string letters = mapping[digits[index] - '0'];
11    for (char c : letters) {
12        current.push_back(c);
13        solve(index + 1, digits, current, mapping, result);
14        current.pop_back();
15    }
16 }
17
18 vector<string> letterCombinations(string digits) {
19     if (digits.empty()) return {};
20
21     vector<string> mapping = {
22         "", "", "abc", "def", "ghi", "jkl",
23         "mno", "pqrs", "tuv", "wxyz"
24     };
25
26     vector<string> result;
27     string current;
28     solve(0, digits, current, mapping, result);
29     return result;
30 }
31
32 int main() {
33     for (string& s : letterCombinations("23")) cout << s << " ";
34     cout << endl;
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. If each digit maps to at most 4 letters, there are at most 4^L combinations for a digit string of length L , each costing $O(L)$ to build, giving

$$O(L \cdot 4^L).$$

Space Complexity. $O(L)$ for the recursion depth.

Key Takeaway

A loop-based recursive branch (“try every choice in this position’s choice set”) is the natural generalization of a hard-coded binary pick/not-pick, and is the more common shape for backtracking problems in practice, since most real choice sets have more than two options. Every remaining backtracking problem in this chapter (Sections 1.18–1.21) uses this loop-based shape, with the choice set determined by the problem’s specific structure (grid moves, board placements, partition points).

1.18 Rat in a Maze

We now begin the chapter's final group: full **constraint-satisfaction backtracking**, where the choice at each step is a move on a grid or board, and a choice already made can block later choices in ways that must be explicitly tracked and undone.

Problem Statement. Given an $n \times n$ binary maze (cells are either open, marked 1, or blocked, marked 0), a rat starts at the top-left cell $(0, 0)$ and wants to reach the bottom-right cell $(n - 1, n - 1)$, moving only through open cells, in any of the four directions (down, left, right, up), without revisiting a cell already used in the current path. Find every path from start to destination, each described as a string of move directions.

Pattern Recognition

“Find all paths through a grid avoiding revisits” is the canonical **grid backtracking** template: at each cell, the choice set is “which of up to 4 neighboring directions to move into next,” each choice must be validated against the grid's boundaries and blocked/visited cells *before* being taken, and – crucially – a cell marked as visited while exploring one direction must be **unmarked** once that direction's exploration fully returns, so that a *different* path (reached via backtracking to an earlier cell) can freely reuse that same cell.

Understanding the problem carefully

At any cell (row, col) during the search, a move into neighbor $(newRow, newCol)$ is valid only if it is within the grid's bounds, the cell there is open (value 1), and it has not already been visited earlier in the *current* path (revisiting would create a cycle, which is never part of a valid simple path). If the move is valid, we mark the destination visited, append the direction's label to the path string, and recurse from there; once that entire recursive exploration finishes (having found every path reachable through this move), we must **unmark** the destination as visited and remove the direction label, so that a sibling direction's exploration – reached by returning to this same cell – does not incorrectly believe this cell is permanently blocked.

Mark Visited Before Recursing, Unmark After Returning

SOLVE($row, col, path$): if (row, col) is the destination: record $path$ (and return, or continue if the destination cell itself could lead further, depending on problem variant). Otherwise, for each of the 4 directions, in a fixed order: compute the neighbor; if it is in bounds, open, and not visited: mark it visited; append the direction's label to $path$; recurse into the neighbor; remove the label from $path$ (undo); unmark the neighbor as visited (undo).

Why unmarking after the recursive call returns is essential for correctness

If a cell, once visited during the exploration of one direction, were *never* unmarked, it would remain permanently blocked for every other path that might also legitimately need to pass through it later – incorrectly preventing valid paths from being discovered. The visited-marking only needs to prevent revisiting *within a single, currently-in-progress path* (to avoid infinite loops/cycles along that one path); it must not persist across different candidate paths, which are explored one at a time, each needing a clean slate. Unmarking immediately after a direction's full recursive exploration returns is exactly what restores that clean slate for the next direction (or for an ancestor's next sibling branch) to use.

Algorithm

1. Mark $(0, 0)$ visited; call SOLVE($0, 0, ""$).

2. SOLVE(*row, col, path*):

- (a) If $(row, col) = (n - 1, n - 1)$: record *path*; return.
- (b) For each direction $(dr, dc, label)$ in $\{(1, 0, D), (0, -1, L), (0, 1, R), (-1, 0, U)\}$:
 - $nr \leftarrow row + dr, nc \leftarrow col + dc$.
 - If nr, nc in bounds, maze cell is open, and not visited: mark visited; append *label* to *path*; recurse SOLVE($nr, nc, path$); remove the appended label (undo); unmark visited (undo).

Dry run

Dry Run. Take the 2×2 maze $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$.

- Start at $(0, 0)$, visited = $\{(0, 0)\}$, *path* = "".
- Try D (down to $(1, 0)$): maze value is 0 (blocked). Invalid, skip.
- Try L (left to $(0, -1)$): out of bounds. Skip.
- Try R (right to $(0, 1)$): open, not visited. Mark visited, *path* = "R". Recurse from $(0, 1)$.
 - Try D (down to $(1, 1)$): open, not visited. Mark visited, *path* = "RD". Recurse from $(1, 1)$: destination reached! Record "RD".
 - Undo: unmark $(1, 1)$, *path* back to "R".
 - Try L (back to $(0, 0)$): visited already, skip.
 - Try U: out of bounds. Skip.
- Undo: unmark $(0, 1)$, *path* back to "".
- Try U (up to $(-1, 0)$): out of bounds. Skip.

The only path found is "RD" (right, then down).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void solve(int row, int col, vector<vector<int>>& maze, int n,
5           vector<vector<bool>>& visited, string& path,
6           vector<string>& result) {
7     if (row == n - 1 && col == n - 1) {
8         result.push_back(path);
9         return;
10    }
11
12    int dr[] = {1, 0, 0, -1};
13    int dc[] = {0, -1, 1, 0};
14    char label[] = {'D', 'L', 'R', 'U'};
15
16    for (int d = 0; d < 4; d++) {
17        int nr = row + dr[d], nc = col + dc[d];
18        if (nr >= 0 && nr < n && nc >= 0 && nc < n &&
19            maze[nr][nc] == 1 && !visited[nr][nc]) {

```

```

20         visited[nr][nc] = true;
21         path.push_back(label[d]);
22
23         solve(nr, nc, maze, n, visited, path, result);
24
25         path.pop_back();
26         visited[nr][nc] = false;
27     }
28 }
29 }
30
31 vector<string> ratInMaze(vector<vector<int>>& maze) {
32     int n = maze.size();
33     vector<string> result;
34     if (maze[0][0] == 0) return result;
35
36     vector<vector<bool>> visited(n, vector<bool>(n, false));
37     visited[0][0] = true;
38     string path;
39     solve(0, 0, maze, n, visited, path, result);
40     return result;
41 }
42
43 int main() {
44     vector<vector<int>> maze = {{1,1},{0,1}};
45     for (string& p : ratInMaze(maze)) cout << p << " ";
46     cout << endl;
47     return 0;
48 }

```

Complexity Analysis

Time Complexity. In the worst case (an open grid with no blocked cells), the number of simple paths can be exponential in n^2 (roughly $O(4^{n^2})$ branching, though heavily pruned by the no-revisit rule in practice); a tight closed-form bound is not generally useful here – what matters is that each cell visit does $O(1)$ work beyond the recursive calls themselves.

Space Complexity. $O(n^2)$ for the visited grid, plus $O(n^2)$ for the maximum recursion depth (a path can visit at most every cell once).

Key Takeaway

“Mark visited before recursing, unmark after returning” is the universal template for any backtracking search over a grid or graph that must avoid revisiting nodes within a single path – it is the direct generalization of the choose/recurse/un-choose rhythm from Section 1.7, applied to spatial moves instead of array-element inclusion. The same template, with different validity checks, drives N-Queens, Sudoku, and Word Search in the sections that follow.

1.19 N-Queens

Problem Statement. Place n queens on an $n \times n$ chessboard so that no two queens attack each other (no two share a row, a column, or a diagonal). Return every distinct valid placement.

Pattern Recognition

“Place n non-attacking items on a board” is a constraint-satisfaction backtracking problem where a powerful structural shortcut is available: since a valid solution can have **at most one queen per row** (two queens in the same row always attack each other), we can place exactly one queen **per row**, in row order, reducing the search from “choose a subset of cells” to “choose one column per row,” checking only column and diagonal conflicts against *already-placed* queens in earlier rows.

Understanding the problem carefully

Because we place queens one complete row at a time, by the time we decide row r ’s queen, rows $0, \dots, r-1$ already have their queens correctly and safely placed; we never need to recheck them against each other again, only check the new candidate column against those already fixed. A candidate column c in row r is safe if no earlier queen shares column c , and no earlier queen lies on either diagonal passing through (r, c) – a queen at (r', c') shares a diagonal with (r, c) exactly when $|r - r'| = |c - c'|$.

Track Occupied Columns and Diagonals for $O(1)$ Safety Checks

Maintain three boolean arrays: `colUsed[c]` (is column c occupied by some already-placed queen), and two diagonal arrays indexed by the constant value along each diagonal direction: `diag1[r + c]` (the “\”-diagonal) and `diag2[r - c + n - 1]` (the “/”-diagonal, offset to stay non-negative). `SOLVE(row)`: if $row = n$: record the current board; return. Otherwise, for each column c from 0 to $n-1$: if none of `colUsed[c]`, `diag1[r + c]`, `diag2[r - c + n - 1]` are set: place the queen, mark all three arrays, recurse `SOLVE(row + 1)`, then unmark all three (undo) and remove the queen.

Why diagonal membership reduces to a single sum or difference

Every cell on the same “\”-diagonal (running from upper-left to lower-right) shares the same value of $row + col$; every cell on the same “/”-diagonal (lower-left to upper-right) shares the same value of $row - col$. This means checking “does any earlier queen share a diagonal with (r, c) ” reduces to a single array lookup at index $r + c$ (or $r - c$, shifted to stay a valid non-negative array index) rather than scanning every previously placed queen and computing $|r - r'|$ versus $|c - c'|$ each time – turning an $O(n)$ safety check into an $O(1)$ one, and the whole algorithm’s per-placement cost from $O(n)$ down to $O(1)$.

Algorithm

1. Initialize `colUsed`, `diag1`, `diag2` as all-false boolean arrays of appropriate sizes.
2. `SOLVE(row)`:
 - (a) If $row = n$: record the board; return.
 - (b) For c from 0 to $n-1$: if `colUsed[c]`, `diag1[row + c]`, and `diag2[row - c + n - 1]` are all false: place the queen at (row, c) ; set all three arrays true; recurse `SOLVE(row + 1)`; set all three arrays back to false (undo); remove the queen.
3. Call `SOLVE(0)`.

Dry run

Dry Run. Take $n = 4$.

The search places a queen in row 0, tries each column, and recurses; the only two solutions (up to the board's symmetry) for $n = 4$ are columns $[1, 3, 0, 2]$ and $[2, 0, 3, 1]$ (queen in row 0 at column 1 or 2 respectively, with the rest determined by the safety constraints). Tracing column 1 for row 0: row 1's columns 0, 1, 2 are all unsafe (column 1 taken, or diagonal conflicts), leaving only column 3 safe; row 2's only safe column, given rows 0, 1, is 0; row 3's only safe column, given rows 0, 1, 2, is 2 – yielding the complete solution $[1, 3, 0, 2]$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void solve(int row, int n, vector<int>& cols, vector<bool>& colUsed,
5            vector<bool>& diag1, vector<bool>& diag2,
6            vector<vector<int>>& result) {
7      if (row == n) {
8          result.push_back(cols);
9          return;
10     }
11     for (int c = 0; c < n; c++) {
12         int d1 = row + c, d2 = row - c + n - 1;
13         if (colUsed[c] || diag1[d1] || diag2[d2]) continue;
14
15         cols[row] = c;
16         colUsed[c] = diag1[d1] = diag2[d2] = true;
17
18         solve(row + 1, n, cols, colUsed, diag1, diag2, result);
19
20         colUsed[c] = diag1[d1] = diag2[d2] = false;
21     }
22 }
23
24 vector<vector<int>> solveNQueens(int n) {
25     vector<vector<int>> result;
26     vector<int> cols(n, -1);
27     vector<bool> colUsed(n, false), diag1(2 * n, false), diag2(2 * n,
28         false);
29
30     solve(0, n, cols, colUsed, diag1, diag2, result);
31     return result;
32 }
33
34 int main() {
35     auto solutions = solveNQueens(4);
36     cout << "Number of solutions: " << solutions.size() << endl;
37     for (auto& sol : solutions) {
38         for (int c : sol) cout << c << " ";
39         cout << endl;
40     }
41     return 0;

```

Complexity Analysis

Time Complexity. Bounded by $O(n!)$ in the absolute worst case (one queen per row, up to n column choices each, shrinking as constraints accumulate), though the $O(1)$ safety check via the tracking arrays makes each individual placement attempt fast; the true practical running time is much smaller due to aggressive pruning from the column/diagonal constraints. **Space Complexity.** $O(n)$ for the tracking arrays and the recursion depth.

Key Takeaway

When a backtracking constraint can be reduced to membership in a small number of equivalence classes (here, columns and the two diagonal directions), maintain one boolean array per class for $O(1)$ membership checks, instead of recomputing the constraint from scratch by scanning all previously placed items on every attempt. This “maintain auxiliary used/unused trackers, mark and unmark them in lockstep with placing and removing a choice” idea generalizes directly to Sudoku’s row/column/box constraints in the next section.

1.20 Sudoku Solver

Problem Statement. Given a partially filled 9×9 Sudoku board (empty cells marked ‘.’), fill every empty cell with a digit from 1 to 9 so that every row, every column, and every one of the nine 3×3 boxes contains each digit exactly once. It is guaranteed the input has a unique solution.

Pattern Recognition

This closing backtracking problem combines two ideas already established: N-Queens’s (Section 1.19) “track constraint membership in auxiliary structures for fast checks” (generalized here to three overlapping constraints – row, column, and box – instead of just column and two diagonals), and a new control-flow idea needed because this problem wants **any one** valid completion, not *every* one: the recursion must **return a boolean** signaling success, so that the instant a complete valid filling is found anywhere in the search, every enclosing call can immediately stop trying further alternatives and propagate that success straight back to the top.

Understanding the problem carefully

Scan the board for the next empty cell (in, say, row-major order). For that cell, try each digit 1 through 9; a digit is valid if it does not already appear elsewhere in the same row, the same column, or the same 3×3 box. If a valid digit is found, place it and recursively attempt to solve the rest of the board. If that recursive attempt *succeeds* (returns **true**, meaning a full valid solution was found somewhere beneath this choice), we are done – stop trying further digits at this cell and propagate **true** upward immediately. If it *fails* (returns **false**), undo this digit and try the next one. If no digit from 1 to 9 works at this cell, this entire branch is a dead end – return **false**, triggering the caller to backtrack and try a different digit at an earlier cell.

Boolean-Returning Backtracking: Stop on First Success

SOLVE(board): find the next empty cell (*row, col*); if none exists, the board is completely and validly filled – return **true** (base case, success). Otherwise, for d from 1 to 9: if placing d at (*row, col*) is valid: place it; if **SOLVE(board)** returns **true**: return **true** immediately (propagate success, no undo needed – this digit is part of the final solution). Otherwise:

remove d (undo) and try the next digit. If every digit from 1 to 9 has been tried without success: return **false**.

Why a boolean return value, with immediate propagation, is the key new idea

Section 1.19 and earlier sections collected *all* valid solutions, so every branch had to be explored regardless of earlier successes. Sudoku, by contrast, only needs *one* valid completion (and the problem guarantees exactly one exists), so the moment a recursive call confirms a digit choice leads all the way to a complete, valid board, there is no reason to keep searching – every enclosing call should immediately stop and also report success, unwinding the entire call stack without exploring any further alternatives. This is exactly Section 1.8’s short-circuit-on-success idea, now applied to a problem where the “success” condition is “the rest of the board can be completed,” checked recursively rather than as a simple base-case comparison.

Algorithm

1. SOLVE(*board*):
 - (a) Find the next empty cell (*row, col*); if none, return **true**.
 - (b) For d from 1 to 9:
 - If d is valid at (*row, col*) (no conflict in its row, column, or 3×3 box): place d . If SOLVE(*board*) returns **true**: return **true**. Else: remove d (undo).
 - (c) Return **false** (no digit worked here; trigger backtracking).

Dry run

Dry Run. A small conceptual trace: suppose cell (0,2) is empty, and digits 1–5 are already ruled out by its row/column/box, but 6 is valid. We place 6 and recurse to the next empty cell. Several cells later, suppose a dead end is reached where no digit from 1–9 is valid at some cell (4,7) – this returns **false**, which propagates back to the most recent choice point (perhaps the placement at (3,5)), forcing it to try its *next* candidate digit instead of the one that led to this dead end. This unwind-and-retry continues until either a different early choice avoids the dead end entirely (and the search eventually completes successfully, propagating **true** all the way back to the top), or – if the puzzle is unsolvable – every possibility is exhausted and the top-level call returns **false**.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool isValid(vector<vector<char>>& board, int row, int col, char d) {
5      for (int i = 0; i < 9; i++) {
6          if (board[row][i] == d) return false;
7          if (board[i][col] == d) return false;
8          if (board[3*(row/3) + i/3][3*(col/3) + i%3] == d) return false;
9      }
10     return true;
11 }
12
13 bool solve(vector<vector<char>>& board) {
14     for (int row = 0; row < 9; row++) {

```

```

15     for (int col = 0; col < 9; col++) {
16         if (board[row][col] != '.') continue;
17
18         for (char d = '1'; d <= '9'; d++) {
19             if (isValid(board, row, col, d)) {
20                 board[row][col] = d;
21                 if (solve(board)) return true;
22                 board[row][col] = '.';
23             }
24         }
25         return false; // no digit worked at this empty cell
26     }
27 }
28 return true; // no empty cell left: solved
29 }
30
31 int main() {
32     vector<vector<char>> board = {
33         {'5','3','.','.','.','7','.','.','.','.'},
34         {'6','.','.','.','1','9','5','.','.','.'},
35         {'.','9','8','.','.','.','.','6','.','.'},
36         {'8','.','.','.','.','6','.','.','.','3'},
37         {'4','.','.','.','8','.','3','.','.','1'},
38         {'7','.','.','.','.','2','.','.','.','6'},
39         {'.','6','.','.','.','.','.','2','8','.'},
40         {'.','.', '.', '.', '4', '1', '9', '.', '.', '5'},
41         {'.','.', '.', '.', '8', '.', '.', '7', '9'}
42     };
43
44     solve(board);
45
46     for (auto& row : board) {
47         for (char c : row) cout << c << " ";
48         cout << endl;
49     }
50     return 0;
51 }

```

Complexity Analysis

Time Complexity. Exponential in the worst case, roughly $O(9^m)$ where m is the number of empty cells (each could in principle try 9 digits), though the row/column/box validity checks prune the vast majority of branches in practice, making real Sudoku puzzles solve very quickly. **Space Complexity.** $O(m)$ for the recursion depth (at most one call per empty cell along the current path), plus $O(1)$ extra for the *isValid* check (which scans a fixed 9 cells each for row, column, and box).

Key Takeaway

Boolean-returning backtracking – where finding *one* success anywhere in a subtree should immediately propagate **true** all the way up, aborting every remaining sibling branch – is the right shape whenever a problem asks for any single valid solution (or asks whether one exists at all), as opposed to enumerating every solution (N-Queens) or counting them. Recognizing which of these three modes (generate all, exists, count) a backtracking problem needs, exactly as flagged back in Section 1.9's takeaway, remains the first and most important design decision.

1.21 Word Search

Problem Statement. Given an $m \times n$ grid of characters and a string *word*, determine whether *word* can be formed by a path of **sequentially adjacent** cells (horizontally or vertically neighboring, not diagonal), where the **same cell may not be used more than once** within the path.

Pattern Recognition

This is Rat in a Maze's (Section 1.18) grid backtracking template, combined with Sudoku's (Section 1.20) boolean-return, short-circuit-on-success control flow: we need a path through the grid (mark-before-recurse, unmark-after-return, exactly as in the maze), but we only need to know *whether one exists* matching the word, so the search should stop the instant a complete match is found, exactly as Sudoku stops the instant a full board is completed.

Understanding the problem carefully

Try starting the search from every cell that matches *word*'s first character (since the path could begin anywhere in the grid). From a starting cell, the path must match *word* character by character, moving to an adjacent, not-yet-used cell at each step; the search succeeds the moment all of *word*'s characters have been matched, and fails along a particular path the moment a required character cannot be matched by any available neighbor.

Boolean Grid Backtracking, with In-Place Visited Marking

SOLVE(*row*, *col*, *index*): if *index* = |*word*|: return **true** (every character matched). If out of bounds, or *grid*[*row*][*col*] $\neq$ *word*[*index*], or this cell is already used in the current path: return **false**. Otherwise: temporarily mark this cell as used (a common space-saving trick: overwrite the grid cell itself with a sentinel character unlikely to match anything, rather than maintaining a separate visited grid). Try all four neighboring directions, recursing with *index* + 1; if *any* direction returns **true**: restore the cell and return **true** immediately. If none do: restore the cell and return **false**.

Why overwriting the cell in place is a safe substitute for a separate visited array

Since the grid cell's original character is only needed for the *duration* of exploring paths through it, and we always restore it before returning control to the caller (whether the exploration succeeded or failed), no information is permanently lost – by the time any sibling branch or ancestor call inspects this cell again, it has already been restored to its original character. This avoids allocating an entire second $m \times n$ boolean grid purely to track visited cells, at the cost of briefly mutating the input grid during the search (acceptable as long as the grid is restored to its original state by the time the overall function returns, which the undo step guarantees).

Algorithm

1. For every cell (*row*, *col*) with *grid*[*row*][*col*] = *word*[0]: if SOLVE(*row*, *col*, 0) returns **true**: the word exists; return **true**.
2. SOLVE(*row*, *col*, *index*):
 - (a) If *index* = |*word*|: return **true**.
 - (b) If out of bounds, or character mismatch, or this exact cell is the sentinel (already in use): return **false**.

- (c) Save the original character; overwrite the cell with a sentinel.
- (d) For each of the 4 directions: if `SOLVE(newRow, newCol, index + 1)` returns `true`: restore the cell; return `true`.
- (e) Restore the cell; return `false`.

3. If no starting cell succeeds: return `false`.

Dry run

Dry Run. Take the grid $\begin{bmatrix} A & B & C & E \\ S & F & C & S \\ A & D & E & E \end{bmatrix}$, `word = "SEE"`.

- Starting cells matching 'S': (1,0) and (1,3).
- Try (1,0): index 0 matches 'S'. Mark used. Try neighbors for index 1 ('E'): up (0,0) = A, no; down (2,0) = A, no; left out of bounds; right (1,1) = F, no. All fail. Restore (1,0). Return `false`.
- Try (1,3): index 0 matches 'S'. Mark used. Try neighbors for index 1 ('E'): up (0,3) = E, match! Mark used. Try neighbors for index 2 ('E'): up out of bounds; down (back to (1,3), sentinel, fail); left (0,2) = C, no; right out of bounds. All fail from (0,3). Restore (0,3). Try down from (1,3) for index 1: (2,3) = E, match! Mark used. Try neighbors for index 2: up (back to (1,3), sentinel, fail); left (2,2) = E, match! index 2 = 3 = `|word|`? Index becomes 3 which equals `|word|` = 3: return `true` immediately.

The word "SEE" is found via the path (1,3) → (2,3) → (2,2).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool solve(int row, int col, int index, vector<vector<char>>& grid,
5             string& word) {
6      if (index == (int)word.size()) return true;
7
8      int m = grid.size(), n = grid[0].size();
9      if (row < 0 || row >= m || col < 0 || col >= n) return false;
10     if (grid[row][col] != word[index]) return false;
11
12     char original = grid[row][col];
13     grid[row][col] = '#'; // sentinel: marks this cell as in use
14
15     int dr[] = {1, -1, 0, 0};
16     int dc[] = {0, 0, 1, -1};
17     bool found = false;
18     for (int d = 0; d < 4 && !found; d++) {
19         found = solve(row + dr[d], col + dc[d], index + 1, grid, word);
20     }
21
22     grid[row][col] = original; // restore
23     return found;
24 }
25

```

```

26 bool wordSearch(vector<vector<char>>& grid, string word) {
27     int m = grid.size(), n = grid[0].size();
28     for (int r = 0; r < m; r++) {
29         for (int c = 0; c < n; c++) {
30             if (grid[r][c] == word[0] && solve(r, c, 0, grid, word)) {
31                 return true;
32             }
33         }
34     }
35     return false;
36 }
37
38 int main() {
39     vector<vector<char>> grid = {
40         {'A', 'B', 'C', 'E'},
41         {'S', 'F', 'C', 'S'},
42         {'A', 'D', 'E', 'E'}
43     };
44     cout << (wordSearch(grid, "SEE") ? "true" : "false") << endl;
45     return 0;
46 }

```

Complexity Analysis

Time Complexity. $O(mn \cdot 4^L)$, where L is the word's length – mn possible starting cells, each potentially exploring up to 4 directions at every one of L steps. **Space Complexity.** $O(L)$ for the recursion depth (no separate visited grid needed, thanks to the in-place sentinel trick).

Key Takeaway

When a grid-backtracking problem only needs a single existence answer, combine Rat in a Maze's mark/unmark mechanics with Sudoku's short-circuit-on-success boolean propagation – and consider whether the “visited” marking can be folded directly into the grid itself (via a temporary sentinel overwrite) rather than allocating a separate tracking structure, whenever the grid's cells are not otherwise needed during the brief window before they are restored.

1.22 Palindrome Partitioning

Problem Statement. Given a string s , partition it into one or more contiguous substrings such that **every** substring in the partition is itself a palindrome. Return every such partitioning.

Pattern Recognition

“Partition a sequence into pieces, each satisfying some property” introduces a new choice shape: instead of choosing *which elements* to include (as in subsets/combinations) or *which direction* to move (as in grid backtracking), the choice here is **how far the next piece should extend** – a loop over every possible end position for the current piece, starting from wherever the previous piece left off.

Understanding the problem carefully

Starting from some position *start* in the string, the next piece of the partition can end anywhere from *start* itself up to the end of the string – but only positions where the resulting substring

$s[start..end]$ is a palindrome are valid choices. For each valid choice of where this piece ends, the rest of the string (from $end + 1$ onward) must be recursively partitioned the same way; reaching the very end of the string with no characters left to partition marks one complete, valid partitioning.

Loop Over Every Valid Palindromic Prefix from the Current Start

SOLVE($start, current$): if $start = |s|$: record a copy of $current$; return. Otherwise: for end from $start$ to $|s| - 1$: if $s[start..end]$ is a palindrome: append this substring to $current$; recurse SOLVE($end + 1, current$); remove the substring (undo).

Why looping over end positions correctly enumerates every partitioning

Every valid partitioning of $s[start..]$ is uniquely determined by where its *first* piece ends – once that is fixed (to some valid palindromic substring), the rest of the partitioning is exactly a partitioning of whatever remains, which is the same sub-problem at a smaller starting position. By trying *every* valid place the first piece could end, and recursively solving the remainder for each choice, we generate every possible partitioning exactly once, with no overlap between different choices of where the first piece ends (since two different end positions for the first piece necessarily produce different first pieces, hence different overall partitionings).

Algorithm

1. SOLVE($start, current$):
 - (a) If $start = |s|$: record $current$; return.
 - (b) For end from $start$ to $|s| - 1$:
 - If $s[start..end]$ is a palindrome: append it to $current$; recurse SOLVE($end + 1, current$); remove it (undo).
2. Call SOLVE(0, []).

Dry run

Dry Run. Take $s = \text{"aab"}$.

- SOLVE(0, []): $end = 0$: $s[0..0] = \text{"a"}$, palindrome. Append. Recurse SOLVE(1, ["a"]).
 - $end = 1$: $s[1..1] = \text{"a"}$, palindrome. Append. Recurse SOLVE(2, ["a", "a"]).
 - * $end = 2$: $s[2..2] = \text{"b"}$, palindrome. Append. Recurse SOLVE(3, ["a", "a", "b"]):
 $start = 3 = |s|$, record ["a", "a", "b"].
 - Undo. $end = 2$ at the ["a"] level: $s[1..2] = \text{"ab"}$, not a palindrome. Skip.
- Undo back to []. $end = 1$ at the top level: $s[0..1] = \text{"aa"}$, palindrome. Append. Recurse SOLVE(2, ["aa"]).
 - $end = 2$: $s[2..2] = \text{"b"}$, palindrome. Append. Recurse SOLVE(3, ["aa", "b"]): record ["aa", "b"].
- $end = 2$ at the top level: $s[0..2] = \text{"aab"}$, not a palindrome. Skip.

Recorded partitionings: ["a", "a", "b"] and ["aa", "b"] – the only two ways to partition "aab" into all-palindrome pieces.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool isPalindrome(string& s, int lo, int hi) {
5      while (lo < hi) {
6          if (s[lo] != s[hi]) return false;
7          lo++; hi--;
8      }
9      return true;
10 }
11
12 void solve(int start, string& s, vector<string>& current,
13            vector<vector<string>>& result) {
14     if (start == (int)s.size()) {
15         result.push_back(current);
16         return;
17     }
18     for (int end = start; end < (int)s.size(); end++) {
19         if (isPalindrome(s, start, end)) {
20             current.push_back(s.substr(start, end - start + 1));
21             solve(end + 1, s, current, result);
22             current.pop_back();
23         }
24     }
25 }
26
27 vector<vector<string>> palindromePartitioning(string s) {
28     vector<vector<string>> result;
29     vector<string> current;
30     solve(0, s, current, result);
31     return result;
32 }
33
34 int main() {
35     for (auto& partition : palindromePartitioning("aab")) {
36         cout << "[ ";
37         for (string& piece : partition) cout << piece << " ";
38         cout << "] ";
39     }
40     cout << endl;
41     return 0;
42 }

```

Complexity Analysis

Time Complexity. Up to $O(2^n)$ partitionings can exist in the worst case (e.g. a string of all-identical characters, where every position could start a new piece), and checking each candidate substring for the palindrome property costs up to $O(n)$, giving an overall worst case of

$$O(n \cdot 2^n).$$

Space Complexity. $O(n)$ for the recursion depth.

Key Takeaway

“Partition a sequence into pieces satisfying a property” is a third fundamental backtracking choice shape, alongside “include or exclude each element” and “move in one of several directions”: loop over every possible *extent* of the next piece from the current position, recurse on the remainder for each valid extent. This same “loop over partition points” shape reappears, generalized with memoization, in the Matrix Chain Multiplication / Partition DP family covered later in the dynamic programming chapters.

1.23 Word Break

Problem Statement. Given a string s and a dictionary of words $wordDict$, determine whether s can be segmented into a sequence of one or more dictionary words placed consecutively (words may be reused any number of times).

Pattern Recognition

Structurally, this is Palindrome Partitioning’s (Section 1.22) “loop over the next piece’s extent” choice shape, combined with Sudoku’s and Word Search’s boolean short-circuit-on-success control flow – but with one crucial difference that makes this section the deliberate bridge into the dynamic programming chapters that follow: the sub-problem “can $s[start..]$ be segmented” depends only on $start$, and the **same** value of $start$ can be reached through many different earlier choices of piece boundaries – these are **overlapping subproblems**, the defining signature of a problem where memoization will pay off enormously.

Understanding the problem carefully

From position $start$, try every possible end position for the next word: if $s[start..end]$ is in the dictionary, recursively check whether the remainder, $s[end + 1..]$, can also be fully segmented. If reaching the very end of the string is possible through any sequence of such choices, s is breakable. As in Sudoku (Section 1.20), the moment any choice leads to a fully successful segmentation of the remainder, we can stop trying further alternatives and propagate **true** immediately.

Loop Over Dictionary-Matching Prefixes, Short-Circuit on Success

SOLVE($start$): if $start = |s|$: return **true** (the whole string has been consumed). Otherwise, for end from $start$ to $|s| - 1$: if $s[start..end]$ is in $wordDict$ and SOLVE($end + 1$) returns **true**: return **true** immediately. If no choice of end leads to success: return **false**.

Why this recursion, run naively, repeats the same work exponentially often

Consider a string with many ways to reach the same intermediate position $start$ – for instance, $s = \text{"aaaaaaaaaaaaaaaaaaaaab"}$ with dictionary $\{\text{"a"}, \text{"aa"}, \text{"aaa"}\}$: the position right after consuming, say, the first 10 characters can be reached by choosing word lengths $1 + 1 + \dots$, or $2 + 2 + \dots$, or $1 + 2 + 1 + \dots$, and many other combinations – yet SOLVE(10) always asks exactly the same question (“can $s[10..]$ be segmented”) regardless of *which* combination of earlier word choices got us to position 10. A naive recursive implementation recomputes the answer to SOLVE(10) separately, from scratch, every single time any path happens to reach it – and the number of distinct paths reaching a given position can grow exponentially with the string’s length, even though there are only $|s| + 1$ *distinct* values $start$ could ever take.

The bridge to dynamic programming

This is precisely the situation dynamic programming exists to fix: **memoization** caches the result of $\text{SOLVE}(start)$ the first time it is computed, in an array (or hash map) indexed by $start$, and every subsequent call with the same $start$ simply looks up the cached answer in $O(1)$ instead of re-deriving it – collapsing the exponential blowup down to $O(|s|)$ distinct subproblems, each solved once. The recursion's *structure* (the recurrence relating $\text{SOLVE}(start)$ to $\text{SOLVE}(end + 1)$ for various end) does not change at all; only the bookkeeping around it does. This exact transformation – recognize overlapping subproblems in an already-correct recursion, then cache results by the parameters that define each distinct subproblem – is the opening idea of the next chapter, where it is applied systematically across dozens of problems, progressing from this kind of memoized top-down recursion to fully iterative tabulation and, where possible, space-optimized tabulation.

Algorithm

1. $\text{SOLVE}(start)$:
 - (a) If $start = |s|$: return **true**.
 - (b) For end from $start$ to $|s| - 1$: if $s[start..end] \in \text{wordDict}$ and $\text{SOLVE}(end + 1)$: return **true**.
 - (c) Return **false**.
2. Call $\text{SOLVE}(0)$.

Dry run

Dry Run. Take $s = \text{"leetcode"}$, $\text{wordDict} = \{\text{"leet"}, \text{"code"}\}$.

- $\text{SOLVE}(0)$: try $end = 0..3$: $s[0..3] = \text{"leet"}$ is in the dictionary. Check $\text{SOLVE}(4)$.
- $\text{SOLVE}(4)$: try $end = 4..7$: $s[4..7] = \text{"code"}$ is in the dictionary. Check $\text{SOLVE}(8)$.
- $\text{SOLVE}(8)$: $start = 8 = |s|$. Return **true**.
- This propagates straight back: $\text{SOLVE}(4)$ returns **true**; $\text{SOLVE}(0)$ returns **true**.

The string segments as $\text{"leet"} + \text{"code"}$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool solve(int start, string& s, unordered_set<string>& wordSet) {
5     if (start == (int)s.size()) return true;
6
7     for (int end = start; end < (int)s.size(); end++) {
8         string piece = s.substr(start, end - start + 1);
9         if (wordSet.count(piece) && solve(end + 1, s, wordSet)) {
10             return true;
11         }
12     }
13     return false;
14 }
15

```

```
16 bool wordBreak(string s, vector<string>& wordDict) {  
17     unordered_set<string> wordSet(wordDict.begin(), wordDict.end());  
18     return solve(0, s, wordSet);  
19 }  
20  
21 int main() {  
22     vector<string> wordDict = {"leet", "code"};  
23     cout << (wordBreak("leetcode", wordDict) ? "true" : "false") <<  
24         endl;  
25     return 0;  
}
```

Complexity Analysis

Time Complexity. $O(2^n)$ in the worst case for this naive (unmemoized) version, due to the overlapping-subproblem blowup described above. With memoization keyed by *start* (an array or hash map of size $O(n)$, each computed once in $O(n)$ work to try all *end* positions), this drops to $O(n^2)$. **Space Complexity.** $O(n)$ for the recursion depth, plus $O(n)$ for the memoization cache once added.

Key Takeaway

This closing problem of the Recursion chapter is a deliberate hinge: everything about its recursive structure is already correct and familiar from this chapter's earlier sections, but its overlapping subproblems expose recursion's biggest weakness – redundant recomputation – setting up exactly the problem that memoization and tabulation, covered next, are designed to solve. Carry this recursion forward unchanged; only the bookkeeping around it is about to be upgraded.